New Grad / Entry-Level

Software Engineer · AI Engineer · Interview Prep MEGA

NVIDIA · AMD · Google · Meta · Microsoft · Apple · Netflix
Palo Alto · AppZen · Intuit · CrowdStrike · ServiceNow

Personalized for: Shivani Nandakumar

BS CS UC Riverside (Dec 2025) · MS expected March 2027 · 3.86 GPA

v1.0 · May 2026

Total Rounds	10
Total Q&A	115+
Target Roles	NCG / Entry SWE · AI Engineer · ML Engineer
Companies	NVIDIA, AMD, Google, Meta, MS, Apple, Netflix, +5
Coding Focus	Python (deep dive), DS&A, GPU/CUDA experience
Resume Edge	CUDA 12x · Delta co-op · ALS recommender · NLP

How to Use This PDF

This doc is built for one purpose: getting **you** through new-grad SWE/AI Engineer interviews at top tech companies. Every answer maps to your real resume — your CUDA 12x speedup project, your Delta co-op work, your Spark MLlib recommender, your PyLucene BM25 search engine, your TextBlob NLP feature at LifeStages.

The 10 Rounds — typical NCG/entry-level interview process:

1. Recruiter Screen (30 min) — fit, comp, work auth, why-us
2. Online Assessment (60-120 min) — HackerRank, 1-3 coding problems
3. Tech Phone Screen (45-60 min) — 1-2 coding + resume deep-dive
4. Onsite Coding Round 1 (60 min) — medium DS&A
5. Onsite Coding Round 2 (60 min) — DS deep-dive + conceptual
6. System Design (45-60 min) — NCG-friendly designs
7. AI/ML Domain Round (45-60 min) — ML fundamentals + your projects
8. Behavioral (45 min) — STAR stories from your real work
9. Hiring Manager (45-60 min) — culture, growth, alignment
10. Comprehensive Python (45-60 min) — language fluency

Plus: Workflows section (debug playbooks) + Comprehensive Cheatsheet appendix.

Study plan (assumes 3 weeks before interviews):
Week 1: Rounds 2, 4, 5 — code daily. Solve LeetCode patterns from the cheatsheet.
Week 2: Rounds 6, 7, 10 — system design + ML + Python deep concepts.
Week 3: Rounds 1, 3, 8, 9 — behavioral and conversational. Speak STAR stories out loud.
Day before: Cheatsheet review only.

The ONE rule: Every story you tell should map to a REAL project from your resume. CUDA BFS (12x), Delta LEAP parser, ALS+KMeans anime recommender, PyLucene Reddit search, TextBlob journaling at LifeStages. These are your gold. Use them.

Table of Contents

Round 1: Recruiter Screen / Phone Screen (30 min)

First conversation — almost always a recruiter (sometimes a sourcer or HR coordinator). NOT technical. They confirm: basic fit, work authorization, role interest, location preferences, salary expectations, school + GPA + grad timeline. The goal is to get to round 2. Don't oversell, don't undersell. Be genuine. The recruiter is your advocate inside the company — if they like you, they fight for you.

Recruiter

Q1. Tell me about yourself.

A: 60-90 seconds. Structure: Education → Most relevant experience → Why this company. Sample: 'I'm Shivani Nandakumar — I just finished my BS in Computer Science at UC Riverside with a 3.86 GPA, and I'm continuing into the Master's program with expected graduation March 2027. My background sits at the intersection of systems and applied AI. I've been a Technical Operations Co-op at Delta Air Lines since May 2024, where I build enterprise data automation in Python — parsing complex XML BoMs for the LEAP engine, applying ML categorization, and visualizing component lineage across thousands of aircraft parts. Outside of Delta, my standout project was a parallel BFS maze solver in CUDA that hit 12x speedup over CPU by tuning thread block sizing and memory access patterns. I'm passionate about [COMPANY] because of [specific reason — Google's scale, NVIDIA's GPU stack, Meta's product impact, etc.], and I'm excited about new-grad roles where I can grow as a systems-and-AI engineer.'

■ ***Tip:*** *Tailor the 'why' to each company. NVIDIA: GPU and AI infrastructure. Google: scale + breadth. Meta: product impact + scale. Apple: design + privacy. Netflix: streaming + ML. Always end with WHY THIS COMPANY.*

Recruiter

Q2. Why [Company]?

A: Be specific. Avoid 'because you're a great company.' Templates by company: NVIDIA → 'I built a CUDA project that taught me how much performance comes from understanding the full GPU stack — NVIDIA is where the future of AI compute is built.' Google → 'I want to work at the intersection of search, AI, and scale. My information retrieval project with PyLucene + BM25 was a tiny version of what Google does at planetary scale.' Meta → 'My NLP project at LifeStages exposed me to social/connection apps. Meta operates the platform layer that millions of those apps depend on.' Microsoft → 'Azure + GitHub + Copilot is the modern developer stack — I'd love to be on the inside of that platform.' Apple → 'I care about design and privacy. Apple's combo of hardware + software + first-party AI is unique.' Netflix → 'My anime recommendation system used ALS, KMeans, and a content lake — Netflix invented this pattern at scale.' Palo Alto Networks → 'Security is moving from add-on to first-class. My OS coursework taught me how attackers think at the kernel level.' AppZen → 'I want to apply NLP and ML to real workflows like AP automation — your platform is the gold standard.' Intuit → 'Tax and small business is one of the highest-impact applied-AI surfaces — TurboTax and QuickBooks ML is shipping every day.' CrowdStrike → 'Falcon is the modern security platform. EDR + threat hunting + ML scoring is exactly the systems-meets-AI work I want.' ServiceNow → 'Workflow automation + AI = the enterprise modernization story. Now Assist is one of the most-deployed enterprise AI products.'

■ ***Tip:*** *Mention ONE specific NVIDIA product (Triton, CUDA), Google product (Gemini, Search), Meta product (Llama, Threads), etc. — proves you've researched, not just applied broadly.*

Recruiter

Q3. Walk me through your resume.

A: Reverse chronological, 2-3 minutes max. 'Currently I'm a Technical Operations Co-op at Delta Air Lines, hybrid in Atlanta, started May 2024. I'm building the MRO Components Capability List — Delta's enterprise tracking system for aircraft part certifications. My main project is parsing complex XML BoMs for the LEAP engine using Python, Pandas, BeautifulSoup, NumPy, and scikit-learn, then automating hierarchy generation across thousands of serialized parts. Before that, I was a Software Engineering Intern at LifeStages, contributing to Align — a mental wellness app in Flutter + Python Flask. I built an NLP-based journaling feature using TextBlob for emotional tone categorization. On the academic side: BS in Computer Science from UC Riverside, December

2025, 3.86 GPA. Now in the Master's program, expected March 2027. Standout projects: a parallel BFS maze solver in CUDA hitting 12x speedup over CPU; an anime recommendation system using Spark MLlib ALS plus KMeans on a 17K-title dataset with a Flask + React frontend; a Reddit search engine using PyLucene with BM25 ranking and NLP preprocessing.'

■ ***Tip:*** *Have both a 60-second AND a 2-minute version. Practice OUT LOUD — silently rehearsing is not the same.*

Recruiter

Q4. What's your work authorization status?

A: Brief and direct. If F-1 with OPT: 'I'm on F-1 student status. I'll have OPT eligibility after the MS, and as a STEM degree I qualify for the 24-month STEM OPT extension. I'd be open to H-1B sponsorship discussions when that aligns with my timeline.' If citizen/PR: 'I'm a US citizen / permanent resident — no sponsorship needed.' Be honest. Don't apologize.

■ ***Tip:*** *Most top companies sponsor for new grads at large scale. NVIDIA, Google, Microsoft, Apple, Meta all sponsor heavily. Some smaller companies (AppZen, ServiceNow) may have constrained budgets — check Glassdoor.*

Recruiter

Q5. What are your salary expectations?

A: DON'T name a single number — name a range, anchored on data. 'Based on levels.fyi and Handshake data for new-grad SWE roles, I'm seeing total comp ranges of $140K to $230K for new grads at top tech, depending on location and tier. I'd target the middle to upper half of that range — base around $130-160K with appropriate equity and signing. But the specifics depend on the package — what range does [Company] typically offer for this role?' If the recruiter pushes for a single number, give the bottom of your acceptable range — leaves room to negotiate up.

■ ***Tip:*** *NEW GRAD BENCHMARKS (2026, US, total comp first year): Google L3/4: $180-230K. Meta E3/4: $180-230K. Apple ICT2/3: $170-220K. Microsoft 59/60: $160-200K. NVIDIA: $180-230K. Netflix L4: $300K+ (cash heavy, no RSU). Smaller (AppZen, ServiceNow, Intuit): $130-180K. CrowdStrike: $150-200K. Palo Alto: $160-220K. ALWAYS research the specific company's levels.fyi before the call.*

Recruiter

Q6. What's your timeline? When can you start?

A: 'I'm currently in the MS program at UC Riverside, with expected completion March 2027. I'm targeting a start date that aligns with that — summer 2027 for full-time. I'm also open to internship or co-op opportunities through MS — those would let me start sooner and ramp before full-time.' Adjust to your actual flexibility.

■ ***Tip:*** *If you're open to part-time during MS or earlier start with co-op, say so. Some companies (NVIDIA, Microsoft, Google) hire MS students for full-time before graduation if you can split time.*

Recruiter

Q7. Are you interviewing with other companies?

A: Honest but vague. 'Yes, I'm in active conversations with several other top tech companies — I'm being selective and applying mainly to roles that combine systems engineering with applied AI. [Company] is high on my list because of [specific reason].' Don't name specific competitors.

■ ***Tip:*** *Saying 'no, only you' weakens your position. Listing competitors strengthens it but only at high level. NEVER name companies by name to a recruiter.*

Recruiter

Q8. What kind of role are you looking for?

A: 'I'm targeting new-grad Software Engineer or AI Engineer roles where I can contribute to real product code from day one. My strongest areas are Python and C++, with applied AI / NLP background from my coursework and projects. I'm interested in teams that combine systems-level work — like the CUDA project I did — with product impact or ML, similar to what I've been doing at Delta. I'm not locked into one domain; I want to learn from senior engineers and grow.'

■ ***Tip:*** *'Open but with strong preferences' is the right NCG framing. Too narrow = harder to place. Too vague = sounds unfocused.*

Recruiter

Q9. Why are you leaving Delta?

A: Positive framing. 'Delta has been an incredible learning experience — I went from never having parsed enterprise XML data to delivering automation that the team relies on daily. The co-op was always time-bound, and as I finish my MS, I'm ready to commit to a permanent role where I can have multi-year impact on a product. [Company] is exactly that — a place where what I build in year one is still running in year five.'

■ ***Tip:*** *Never bash your current employer. Always pull-toward-new, not push-away-from-old.*

Recruiter

Q10. What are you looking for in your next role?

A: 'Three things. First, technically interesting work — I want problems that stretch me beyond what I've already done. The CUDA project taught me I love systems performance work; the Delta automation taught me I love applied AI on real data. Second, a team that mentors new grads — I want to learn from senior engineers in code review, design docs, and shipping. Third, a culture that ships — I'd rather work on something used by millions than perfect something used by no one.'

■ ***Tip:*** *Three crisp things > five wishy-washy ones. Tie each to a real experience from your resume.*

Recruiter

Q11. What questions do you have for me?

A: Always have 3-5 ready. Recruiter-level questions: (1) 'What does the interview process from here look like — how many rounds, what topics?' (2) 'What teams are hiring new grads for this role? Is there team-matching after offer, or is it pre-assigned?' (3) 'What does the new-grad onboarding program look like — buddy, mentor, training period?' (4) 'How does the team think about MS students vs BS students at the new-grad level?' (5) 'What's the typical timeline from final round to offer?'

■ ***Tip:*** *NEVER say 'no, I'm good.' Even one question is better than zero. Asking shows engagement and helps the recruiter sell you internally.*

Recruiter

Q12. How did you hear about us?

A: Be honest. 'I applied through your careers site after seeing the new-grad SWE posting / I was referred by [person] / I attended your tech talk at UC Riverside / I follow your engineering blog.' If you have a referral, MENTION IT — it gets your resume top-of-pile.

■ ***Tip:*** *Referrals matter enormously at FAANG. If you have one, name them. If you don't, get one for next time.*

Recruiter

Q13. What's your GPA?

A: Direct. 'My undergrad GPA at UC Riverside is 3.86 out of 4.' Don't apologize, don't volunteer extra detail unless asked.

■ ***Tip:*** *3.86 is excellent for top tech (most have implicit 3.5 cutoffs, 3.7+ preferred). Some companies (Meta, Apple, NVIDIA) explicitly ask. Many don't ask but read it from resume.*

Round 2: Online Assessment / HackerRank (60-120 min)

Most top tech companies use online assessments before the phone screen. Format: HackerRank or CodeSignal, 1-3 coding problems, 60-120 minutes. New grad bar = LeetCode Easy/Medium (mostly Medium). Difficulty: 60-70% of candidates fail at this stage. Strategy: read all problems first, attack easiest first to lock in a solve, then go for the harder one. ALWAYS leave 5 minutes to verify edge cases. Code quality matters — write readable Python, use type hints when natural, handle empty input.

OA - Coding

Q1. [Two Pointers] Given a sorted array of integers, return all pairs that sum to a target value.

A: Classic two-pointer on sorted input. O(n) time, O(1) space. The interviewer-favorite — make sure you can write it cleanly.

```
from typing import List, Tuple

def two_sum_sorted(arr: List[int], target: int) -> List[Tuple[int, int]]:
    """
    Sorted array, find all pairs (a, b) where a + b == target.
    Returns pairs as (smaller, larger).
    O(n) time, O(1) extra space (besides output).
    """
    pairs = []
    left, right = 0, len(arr) - 1
    while left < right:
        s = arr[left] + arr[right]
        if s == target:
            pairs.append((arr[left], arr[right]))
            left += 1
            right -= 1
            # Skip duplicates if asked for unique pairs only
            while left < right and arr[left] == arr[left - 1]:
                left += 1
            while left < right and arr[right] == arr[right + 1]:
                right -= 1
        elif s < target:
            left += 1
        else:
            right -= 1
    return pairs

# Test
print(two_sum_sorted([1, 2, 3, 4, 5, 6], 7))
# [(1, 6), (2, 5), (3, 4)]
print(two_sum_sorted([1, 1, 2, 3, 3, 4], 4))
# [(1, 3)] — duplicates skipped
```

■ ***Tip:*** *If they ask the UNSORTED version: use a hash map for O(n) time + O(n) space. Sorted = two pointers. Unsorted = hash.*

OA - Coding

Q2. [Hash Map] Given an array of strings, group anagrams together.

A: Hash by sorted-character signature. O(n × k log k) where k = avg string length.

```
from collections import defaultdict
from typing import List

def group_anagrams(strs: List[str]) -> List[List[str]]:
    """
    Group strings that are anagrams of each other.
    Anagram signature: sorted characters.
    Time: O(n * k log k) where n=count, k=avg length
    Space: O(n * k)
    """
    groups = defaultdict(list)
```

```
    for s in strs:
        key = ''.join(sorted(s))   # signature
        groups[key].append(s)
    return list(groups.values())

# Alternative: char count signature (faster for long strings)
def group_anagrams_count(strs: List[str]) -> List[List[str]]:
    groups = defaultdict(list)
    for s in strs:
        count = [0] * 26
        for c in s:
            count[ord(c) - ord('a')] += 1
        key = tuple(count)   # tuples are hashable
        groups[key].append(s)
    return list(groups.values())

# Test
print(group_anagrams(["eat", "tea", "tan", "ate", "nat", "bat"]))
# [['eat','tea','ate'], ['tan','nat'], ['bat']]
```

■ ***Tip:*** *Count-based key is O(k) per string vs O(k log k) for sorted. Mention both options to interviewer.*

OA - Coding

Q3. [Sliding Window] Find the length of the longest substring without repeating characters.

A: Sliding window with hash set. O(n) time, O(min(n, alphabet)) space.

```
def longest_unique_substring(s: str) -> int:
    """
    Longest substring with all-unique characters.
    Returns the length.
    """
    seen = {}        # char -> last index seen
    left = 0
    best = 0
    for right, c in enumerate(s):
        if c in seen and seen[c] >= left:
            # Repeat inside window — move left past previous occurrence
            left = seen[c] + 1
        seen[c] = right
        best = max(best, right - left + 1)
    return best

# Test
print(longest_unique_substring("abcabcbb"))  # 3 ("abc")
print(longest_unique_substring("bbbbb"))     # 1 ("b")
print(longest_unique_substring("pwwkew"))    # 3 ("wke")
print(longest_unique_substring(""))          # 0
```

■ ***Tip:*** *The trick: 'seen[c] >= left' — only skip if the repeat is INSIDE current window. Without that check, you'd jump back past characters that already left the window.*

OA - Coding

Q4. [BFS] Given a 2D grid of 0s and 1s, find the shortest path from top-left to bottom-right (0=walkable, 1=wall). Can move 4 directions. Return -1 if no path.

A: Your CUDA maze solver background — exactly this problem. Tell the interviewer!

```
from collections import deque
from typing import List

def shortest_path(grid: List[List[int]]) -> int:
    """
    Shortest path in a grid from (0,0) to (m-1, n-1).
    0 = walkable, 1 = wall.
    BFS guarantees shortest path on unweighted grid.
    Time: O(m*n), Space: O(m*n)
    """
    if not grid or not grid[0]:
        return -1
    m, n = len(grid), len(grid[0])
    if grid[0][0] == 1 or grid[m-1][n-1] == 1:
```

```
        return -1

    queue = deque([(0, 0, 1)])      # (row, col, distance)
    visited = {(0, 0)}
    directions = [(-1, 0), (1, 0), (0, -1), (0, 1)]

    while queue:
        r, c, dist = queue.popleft()
        if (r, c) == (m-1, n-1):
            return dist
        for dr, dc in directions:
            nr, nc = r + dr, c + dc
            if (0 <= nr < m and 0 <= nc < n
                    and grid[nr][nc] == 0
                    and (nr, nc) not in visited):
                visited.add((nr, nc))
                queue.append((nr, nc, dist + 1))
    return -1

# Test
grid = [
    [0, 0, 0],
    [1, 1, 0],
    [0, 0, 0]
]
print(shortest_path(grid))   # 5
```

■ ***Tip:*** *Drop the CUDA reference: 'I implemented BFS on a 512x512 grid in CUDA for my parallel computing project, achieving 12x speedup over CPU.' Instant credibility.*

OA - Coding

Q5. [Dynamic Programming] Climb stairs — n stairs, can take 1 or 2 steps at a time. How many distinct ways?

A: Fibonacci pattern. f(n) = f(n-1) + f(n-2). Classic DP starter.

```
def climb_stairs(n: int) -> int:
    """
    Ways to climb n stairs taking 1 or 2 at a time.
    Recurrence: f(n) = f(n-1) + f(n-2), with f(0)=1, f(1)=1.
    Time: O(n), Space: O(1) with two-variable optimization.
    """
    if n <= 1:
        return 1
    prev2, prev1 = 1, 1
    for _ in range(2, n + 1):
        curr = prev1 + prev2
        prev2 = prev1
        prev1 = curr
    return prev1

# Tabulated DP version (O(n) space, more explicit):
def climb_stairs_dp(n: int) -> int:
    if n <= 1:
        return 1
    dp = [0] * (n + 1)
    dp[0] = dp[1] = 1
    for i in range(2, n + 1):
        dp[i] = dp[i-1] + dp[i-2]
    return dp[n]

# Test
print(climb_stairs(5))   # 8
print(climb_stairs(10))  # 89
```

■ ***Tip:*** *Always optimize from O(n) space to O(1) when only the last 2 values matter. Mention both versions.*

OA - Coding

Q6. [String Manipulation] Reverse words in a sentence, preserving word order... no wait, reverse the order. Each word should stay intact, but the sequence of words should be reversed.

```
def reverse_words(s: str) -> str:
    """
    'Hello World Python' -> 'Python World Hello'
    Handles multiple spaces, leading/trailing whitespace.
    """
    # split() with no args splits on any whitespace AND removes empties
    return ' '.join(s.split()[::-1])

# Manual without split (for in-place style):
def reverse_words_manual(s: str) -> str:
    words = []
    word = []
    for c in s:
        if c == ' ':
            if word:
                words.append(''.join(word))
                word = []
        else:
            word.append(c)
    if word:
        words.append(''.join(word))
    return ' '.join(reversed(words))

# Test
print(reverse_words("  Hello   World  Python  "))  # 'Python World Hello'
```

■ ***Tip:*** *The default split() with no args is MAGIC — handles all whitespace edge cases. Mention this explicitly.*

OA - Coding

Q7. [Linked List] Reverse a singly linked list.

A: The canonical interview question. Three-pointer iterative is preferred.

```
from typing import Optional

class ListNode:
    def __init__(self, val=0, next=None):
        self.val = val
        self.next = next

def reverse_list(head: Optional[ListNode]) -> Optional[ListNode]:
    """
    Reverse singly linked list, iterative.
    Time O(n), Space O(1).
    """
    prev = None
    curr = head
    while curr:
        next_node = curr.next   # save next BEFORE breaking link
        curr.next = prev        # reverse the link
        prev = curr             # advance prev
        curr = next_node        # advance curr
    return prev   # new head

# Recursive (elegant but O(n) stack space):
def reverse_list_recursive(head: Optional[ListNode]) -> Optional[ListNode]:
    if not head or not head.next:
        return head
    new_head = reverse_list_recursive(head.next)
    head.next.next = head    # the trick: make next point back
    head.next = None
    return new_head
```

■ ***Tip:*** *Always do iterative first. Mention recursive as 'cleaner but uses stack space, problematic for long lists.'*

OA - Coding

Q8. [Tree] Given a binary tree, return its level-order traversal (BFS).

```
from collections import deque
from typing import Optional, List

class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val = val
```

```
        self.left = left
        self.right = right

def level_order(root: Optional[TreeNode]) -> List[List[int]]:
    """
    BFS level-by-level. Each level returned as a separate list.
    Time O(n), Space O(w) where w = max width.
    """
    if not root:
        return []
    result = []
    queue = deque([root])
    while queue:
        level_size = len(queue)   # capture BEFORE we add children
        current_level = []
        for _ in range(level_size):
            node = queue.popleft()
            current_level.append(node.val)
            if node.left:
                queue.append(node.left)
            if node.right:
                queue.append(node.right)
        result.append(current_level)
    return result

# Test:
#     3
#    / \
#   9  20
#      / \
#     15  7
root = TreeNode(3, TreeNode(9), TreeNode(20, TreeNode(15), TreeNode(7)))
print(level_order(root))   # [[3], [9, 20], [15, 7]]
```

■ ***Tip:*** *The 'level_size = len(queue)' trick is what separates per-level vs flat BFS. Practice this — it's asked everywhere.*

OA - Coding

Q9. [Search] Implement binary search on a sorted array.

A: Should be muscle memory. The off-by-one errors are what they're testing.

```
from typing import List

def binary_search(arr: List[int], target: int) -> int:
    """
    Returns index of target in sorted array, or -1 if not found.
    Time O(log n), Space O(1).
    """
    left, right = 0, len(arr) - 1
    while left <= right:           # <= is critical (not <)
        mid = left + (right - left) // 2   # avoid overflow
        if arr[mid] == target:
            return mid
        elif arr[mid] < target:
            left = mid + 1
        else:
            right = mid - 1
    return -1

# Variant: leftmost occurrence (first index where arr[i] >= target)
def lower_bound(arr: List[int], target: int) -> int:
    left, right = 0, len(arr)      # right = len, not len-1
    while left < right:
        mid = (left + right) // 2
        if arr[mid] < target:
            left = mid + 1
        else:
            right = mid
    return left   # index of first >= target, or len if all smaller

# Python stdlib equivalent: bisect.bisect_left(arr, target)
import bisect
print(bisect.bisect_left([1, 2, 4, 4, 5], 4))   # 2
```

■ ***Tip:*** *Mention bisect module — interviewers love when you know Python stdlib. 'In production I'd use bisect, but here's the from-scratch version.'*

Round 3: Technical Phone Screen (45-60 min)

After OA pass, you talk to an engineer. Shared coding pad (CoderPad, HackerRank Live, or Google Docs). 1-2 coding problems, usually one Easy/Medium warm-up then one Medium. Sometimes also asks about your resume projects. CRITICAL: think out loud, ask clarifying questions, and walk through edge cases BEFORE coding.

Phone Screen

Q1. Walk me through your CUDA project. What did you optimize?

A: This is on YOUR resume — interviewer will absolutely ask. Be ready with depth. 'I implemented a parallel BFS maze solver in CUDA on 512x512 mazes, hitting 12x speedup vs the CPU version. The CPU baseline was sequential C++ BFS using a queue. For CUDA, I represented the frontier as a device-side array, where each thread processed one node from the current frontier and atomically marked its neighbors for the next frontier. Three optimizations mattered most. First, thread block sizing — I benchmarked 64, 128, 256, 512 threads per block, and 256 gave best occupancy for our SM count. Second, memory access — I switched from a struct-of-pointers layout to a flat row-major grid, giving coalesced reads from global memory. Third, load balancing — for early BFS levels the frontier is small and we underutilize the GPU; for late levels it explodes. I split the level traversal so the GPU handles only the dense levels, falling back to CPU for sparse early ones. Trade-off I'd revisit: kernel launch overhead per level is real; a persistent-kernel approach could cut that further.'

■ ***Tip:*** *Be ready for follow-ups: 'What's coalesced memory access? Why does it matter?' 'What's warp divergence?' 'How would you scale this beyond one GPU?' Know your project COLD.*

Phone Screen

Q2. Tell me about a time you debugged a hard problem.

A: STAR framework. Sample: 'During my Delta co-op, the LEAP engine BoM parser was returning malformed hierarchy for about 3% of parts — they showed up as orphans with no parent. Initial assumption: corrupt XML. I dumped the raw XML for affected parts and discovered the data was fine — but our parser was case-sensitive on a part-number field that the upstream system had recently started emitting in lowercase. The bug had been latent for weeks until that schema change. Fix: I normalized to uppercase on parse, added a unit test specifically for case variance, and flagged the upstream schema change to the data team so they could communicate it cross-team. Lesson: don't assume data is the problem just because the code looks right — log the raw input and compare.'

■ ***Tip:*** *Pick a REAL story from Delta or LifeStages. Generic 'I once debugged something' answers fail. The interviewer wants specifics — what tools, what hypothesis, what insight.*

Phone Screen

Q3. [Coding] Given two strings s and t, return true if t is an anagram of s.

A: Warm-up question. Don't overthink.

```
def is_anagram(s: str, t: str) -> bool:
    """
    True if t is an anagram of s.
    Time O(n), Space O(k) where k = distinct chars.
    """
    if len(s) != len(t):
        return False
    # Method 1: char count
    from collections import Counter
    return Counter(s) == Counter(t)

# Method 2: sort and compare (O(n log n) but one-liner)
def is_anagram_sort(s: str, t: str) -> bool:
    return sorted(s) == sorted(t)

# Method 3: manual count (no imports, shows you understand the algorithm)
def is_anagram_manual(s: str, t: str) -> bool:
```

```
    if len(s) != len(t):
        return False
    count = {}
    for c in s:
        count[c] = count.get(c, 0) + 1
    for c in t:
        if c not in count:
            return False
        count[c] -= 1
        if count[c] < 0:
            return False
    return True
```

■ ***Tip:*** *Always offer multiple solutions and discuss trade-offs. 'Counter is cleanest. Sort is shortest. Manual count is best if no imports allowed.' That breadth signals seniority.*

Phone Screen

Q4. [Coding] Find the kth largest element in an unsorted array.

A: Classic. Multiple approaches — discuss trade-offs.

```
from typing import List
import heapq

def kth_largest(nums: List[int], k: int) -> int:
    """
    Find kth largest element. Multiple approaches:

    Approach 1: Sort
    Time: O(n log n)
    Space: O(1) if in-place, O(n) otherwise
    """
    return sorted(nums, reverse=True)[k - 1]

def kth_largest_heap(nums: List[int], k: int) -> int:
    """
    Approach 2: Min-heap of size k
    Time: O(n log k) — better than sort when k << n
    Space: O(k)
    """
    heap = []
    for n in nums:
        heapq.heappush(heap, n)
        if len(heap) > k:
            heapq.heappop(heap)
    return heap[0]   # smallest of the k largest = kth largest

# Python stdlib equivalent (uses introselect, average O(n)):
def kth_largest_lib(nums: List[int], k: int) -> int:
    return heapq.nlargest(k, nums)[-1]

# Approach 3: Quickselect (average O(n), worst O(n^2))
import random
def kth_largest_quickselect(nums: List[int], k: int) -> int:
    """k-th largest = (n-k)-th smallest (0-indexed: n-k)"""
    nums = nums.copy()
    target_idx = len(nums) - k

    def partition(lo, hi):
        pivot = random.randint(lo, hi)
        nums[pivot], nums[hi] = nums[hi], nums[pivot]
        store = lo
        for i in range(lo, hi):
            if nums[i] < nums[hi]:
                nums[store], nums[i] = nums[i], nums[store]
                store += 1
        nums[store], nums[hi] = nums[hi], nums[store]
        return store

    lo, hi = 0, len(nums) - 1
    while lo <= hi:
        p = partition(lo, hi)
        if p == target_idx:
            return nums[p]
```

```
        elif p < target_idx:
            lo = p + 1
        else:
            hi = p - 1
    return -1   # unreachable for valid input
```

■ ***Tip:*** *Discuss: 'Sort is simple but wasteful. Heap is best for k << n. Quickselect is fastest on average but worst-case quadratic — and Python recursion depth limits it.' Junior says 'sort.' Senior says 'depends on k vs n.'*

Phone Screen

Q5. [Coding] Given a string, find the longest palindromic substring.

A: Expand-around-center is the cleanest O(n²) solution. Manacher's algorithm is O(n) but overkill for interviews.

```
def longest_palindrome(s: str) -> str:
    """
    Longest palindromic substring.
    Approach: expand around every possible center.
    n centers for odd-length, n-1 for even-length = 2n-1 total.
    Time: O(n^2). Space: O(1).
    """
    if not s:
        return ""

    def expand(left: int, right: int) -> str:
        while left >= 0 and right < len(s) and s[left] == s[right]:
            left -= 1
            right += 1
        return s[left + 1: right]   # exclusive of failure positions

    best = ""
    for i in range(len(s)):
        # Odd-length palindrome centered at i
        odd = expand(i, i)
        # Even-length palindrome centered between i and i+1
        even = expand(i, i + 1)
        for candidate in (odd, even):
            if len(candidate) > len(best):
                best = candidate
    return best

# Test
print(longest_palindrome("babad"))    # "bab" or "aba"
print(longest_palindrome("cbbd"))     # "bb"
print(longest_palindrome("a"))        # "a"
print(longest_palindrome(""))         # ""

# DP version (O(n^2) but cleaner mental model):
def longest_palindrome_dp(s: str) -> str:
    n = len(s)
    if n == 0:
        return ""
    # dp[i][j] = True if s[i:j+1] is palindrome
    dp = [[False] * n for _ in range(n)]
    start, max_len = 0, 1
    # Length 1: always palindrome
    for i in range(n):
        dp[i][i] = True
    # Length 2
    for i in range(n - 1):
        if s[i] == s[i + 1]:
            dp[i][i + 1] = True
            start, max_len = i, 2
    # Length 3+
    for length in range(3, n + 1):
        for i in range(n - length + 1):
            j = i + length - 1
            if s[i] == s[j] and dp[i + 1][j - 1]:
                dp[i][j] = True
                if length > max_len:
                    start, max_len = i, length
    return s[start: start + max_len]
```

■ ***Tip:*** *Always go with expand-around-center for whiteboard — easier to write correctly. Mention DP as alternative.*

Phone Screen

Q6. [Coding] Implement a function that checks if a string of parentheses is balanced. Handle multiple types: (), {}, [].

A: Stack-based. Standard answer.

```
def is_balanced(s: str) -> bool:
    """
    Returns True if every open bracket has a matching close bracket,
    in correct nesting order.
    Examples: '({[]})' -> True, '([)]' -> False
    Time: O(n). Space: O(n) for the stack.
    """
    pairs = {')': '(', '}': '{', ']': '['}
    stack = []
    for c in s:
        if c in '({[':
            stack.append(c)
        elif c in ')}]':
            if not stack or stack[-1] != pairs[c]:
                return False
            stack.pop()
        # ignore other characters (or fail strictly — discuss)
    return not stack   # all opens must be closed

# Test
print(is_balanced("({[]})"))     # True
print(is_balanced("([)]"))       # False
print(is_balanced("("))          # False
print(is_balanced(""))           # True
print(is_balanced("()()"))       # True
```

■ ***Tip:*** *Ask the interviewer: 'Should non-bracket characters cause failure, or be ignored?' Senior signal.*

Phone Screen

Q7. [Coding] Given a list of integers, find three numbers that sum to zero. Return all unique triplets.

A: Sort + two pointers. Classic 3Sum.

```
from typing import List

def three_sum(nums: List[int]) -> List[List[int]]:
    """
    Find all unique triplets in nums that sum to 0.
    Time: O(n^2). Space: O(1) extra (besides output).
    Pattern: fix one element, two-pointer the rest.
    """
    nums = sorted(nums)
    result = []
    n = len(nums)

    for i in range(n - 2):
        # Skip duplicate first elements
        if i > 0 and nums[i] == nums[i - 1]:
            continue
        # Optimization: if smallest possible sum > 0, no triplet works
        if nums[i] > 0:
            break

        left, right = i + 1, n - 1
        target = -nums[i]

        while left < right:
            s = nums[left] + nums[right]
            if s == target:
                result.append([nums[i], nums[left], nums[right]])
                # Skip duplicates on both sides
                while left < right and nums[left] == nums[left + 1]:
                    left += 1
                while left < right and nums[right] == nums[right - 1]:
                    right -= 1
```

```
                left += 1
                right -= 1
            elif s < target:
                left += 1
            else:
                right -= 1
    return result

# Test
print(three_sum([-1, 0, 1, 2, -1, -4]))   # [[-1, -1, 2], [-1, 0, 1]]
```

■ ***Tip:*** *The duplicate-skip logic trips up most candidates. Practice this exact problem multiple times.*

Phone Screen

Q8. [Coding] Given a 2D matrix, rotate it 90 degrees clockwise in-place.

A: Transpose then reverse rows. Elegant.

```
from typing import List

def rotate(matrix: List[List[int]]) -> None:
    """
    Rotate matrix 90 degrees clockwise IN-PLACE.
    Approach: transpose along main diagonal, then reverse each row.
    Time O(n^2). Space O(1).
    """
    n = len(matrix)

    # Step 1: Transpose (swap matrix[i][j] with matrix[j][i])
    for i in range(n):
        for j in range(i + 1, n):
            matrix[i][j], matrix[j][i] = matrix[j][i], matrix[i][j]

    # Step 2: Reverse each row
    for row in matrix:
        row.reverse()

# Visualization:
# Original:    Transposed:    Reversed rows:
# 1 2 3        1 4 7          7 4 1
# 4 5 6   ->   2 5 8     ->   8 5 2
# 7 8 9        3 6 9          9 6 3

# Test
matrix = [
    [1, 2, 3],
    [4, 5, 6],
    [7, 8, 9]
]
rotate(matrix)
print(matrix)   # [[7,4,1],[8,5,2],[9,6,3]]
```

■ ***Tip:*** *If asked counter-clockwise: reverse each row first, then transpose. Or transpose then reverse each column.*

Phone Screen

Q9. Talk about a project you're most proud of.

A: Pick the one with most depth. Sample for CUDA: 'My most complex project was the CUDA BFS maze solver. I chose it because the problem looked simple — BFS is a freshman algorithm — but parallelizing it on a GPU forced me to learn how memory hierarchy, thread scheduling, and warp execution actually work. The thing I'm proudest of isn't the 12x speedup — it's that I instrumented the kernel with NVIDIA Nsight, found that my early version was memory-bound from uncoalesced access, restructured the grid layout, and watched the speedup jump from 4x to 12x. That's the muscle I want to keep building: not just write code, but PROFILE it and improve.'

■ ***Tip:*** *End every project story with what you LEARNED, not just what you DID. Senior signal.*

Round 4: Onsite Coding Round 1 (60 min)

First of usually two onsite coding rounds. Format: 1-2 medium LeetCode-style problems in a shared editor, with the interviewer. Expectations: think out loud, ask clarifying questions, discuss multiple approaches, write CORRECT code, test it against edge cases. The interviewer is grading reasoning, not just final answer. NEW GRAD bar: medium problems solved cleanly in 30-40 minutes each.

Onsite Coding

Q1. [Medium] Given the head of a linked list, determine if it has a cycle. Bonus: return the node where the cycle begins.

A: Floyd's tortoise-and-hare. The bonus part (cycle start) catches juniors.

```
from typing import Optional

class ListNode:
    def __init__(self, val=0, next=None):
        self.val = val
        self.next = next

def has_cycle(head: Optional[ListNode]) -> bool:
    """
    Floyd's algorithm: slow moves 1 step, fast moves 2 steps.
    If they meet, there's a cycle. If fast hits None, no cycle.
    Time O(n), Space O(1).
    """
    slow = fast = head
    while fast and fast.next:
        slow = slow.next
        fast = fast.next.next
        if slow == fast:
            return True
    return False

def cycle_start(head: Optional[ListNode]) -> Optional[ListNode]:
    """
    Find the node where the cycle begins.

    Trick: after slow and fast meet, reset slow to head.
    Move both 1 step at a time. Where they meet = cycle start.

    Why this works:
    - Let L = distance from head to cycle start
    - Let C = cycle length
    - Let m = distance from cycle start to meeting point
    - slow traveled L + m steps; fast traveled 2(L + m)
    - fast lapped slow some k times: 2(L + m) = L + m + k*C
    - So L + m = k*C, which means L = k*C - m
    - From meeting point, moving (k*C - m) steps lands at cycle start
    - From head, L steps also lands at cycle start. They meet.
    """
    slow = fast = head
    while fast and fast.next:
        slow = slow.next
        fast = fast.next.next
        if slow == fast:
            # Phase 2: find cycle start
            slow = head
            while slow != fast:
                slow = slow.next
                fast = fast.next
            return slow
    return None
```

■ ***Tip:*** *Floyd's is interview gold. Be able to derive the cycle-start math out loud. Alternative: hash set of visited nodes — O(n) space but easier to explain.*

Onsite Coding

Q2. [Medium] Given a binary tree, return the rightmost node visible from the right side. (Right-side view.)

A: BFS with last-of-each-level. Or DFS preferring right-first.

```
from collections import deque
from typing import Optional, List

class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val = val
        self.left = left
        self.right = right

def right_side_view(root: Optional[TreeNode]) -> List[int]:
    """
    Return the rightmost node value at each level.
    BFS approach — last popped at each level wins.
    Time O(n), Space O(w).
    """
    if not root:
        return []
    result = []
    queue = deque([root])
    while queue:
        level_size = len(queue)
        for i in range(level_size):
            node = queue.popleft()
            # Last node at this level
            if i == level_size - 1:
                result.append(node.val)
            if node.left:
                queue.append(node.left)
            if node.right:
                queue.append(node.right)
    return result

# DFS approach (right-first, capture first node seen at each depth):
def right_side_view_dfs(root: Optional[TreeNode]) -> List[int]:
    result = []
    def dfs(node, depth):
        if not node:
            return
        if depth == len(result):     # first node seen at this depth
            result.append(node.val)
        dfs(node.right, depth + 1)   # CRITICAL: right first
        dfs(node.left, depth + 1)
    dfs(root, 0)
    return result
```

■ ***Tip:*** *BFS is more intuitive for this problem. DFS works but requires right-first traversal — easy to get wrong.*

Onsite Coding

Q3. [Medium] Given an array, find the contiguous subarray with the largest sum. (Kadane's algorithm.)

A: DP classic — Kadane's algorithm. O(n) time, O(1) space.

```
from typing import List

def max_subarray(nums: List[int]) -> int:
    """
    Largest contiguous subarray sum.
    Kadane's: at each position, decide — extend previous OR start new.
    Time O(n), Space O(1).
    """
    if not nums:
        return 0   # ask interviewer: empty input behavior?

    current_sum = best_sum = nums[0]
    for num in nums[1:]:
        # Extend the current run, or start fresh from here
```

```
        current_sum = max(num, current_sum + num)
        best_sum = max(best_sum, current_sum)
    return best_sum

# Variant: return the actual subarray, not just the sum
def max_subarray_indices(nums: List[int]) -> List[int]:
    if not nums:
        return []
    current_sum = best_sum = nums[0]
    start = end = best_start = best_end = 0
    for i in range(1, len(nums)):
        if nums[i] > current_sum + nums[i]:
            current_sum = nums[i]
            start = i
        else:
            current_sum += nums[i]
        if current_sum > best_sum:
            best_sum = current_sum
            best_start = start
            best_end = i
    return nums[best_start: best_end + 1]

# Test
print(max_subarray([-2, 1, -3, 4, -1, 2, 1, -5, 4]))   # 6 ([4,-1,2,1])
print(max_subarray([1]))                                # 1
print(max_subarray([-1, -2, -3]))                       # -1 (still pick best)
```

■ ***Tip:*** *Be ready for follow-ups: 'What if all numbers are negative?' (return the largest single number, not 0). 'What's the time/space complexity?' (O(n)/O(1)). 'Can you do it for 2D?' (much harder — Kadane on each strip).*

Onsite Coding

Q4. [Medium] Given an array of integers and a target k, return the maximum sum of any contiguous subarray of size k.

A: Sliding window of fixed size. Pattern shows up everywhere.

```
from typing import List

def max_sum_window(nums: List[int], k: int) -> int:
    """
    Max sum of any k-sized contiguous subarray.
    Sliding window of fixed size.
    Time O(n), Space O(1).
    """
    if len(nums) < k:
        return 0  # or raise — clarify with interviewer

    # Initial window sum
    window_sum = sum(nums[:k])
    best = window_sum

    # Slide the window
    for i in range(k, len(nums)):
        window_sum += nums[i] - nums[i - k]   # add new, remove old
        best = max(best, window_sum)
    return best

# Test
print(max_sum_window([1, 4, 2, 10, 23, 3, 1, 0, 20], 4))   # 39 (10+23+3+1+0+20... no wait)
# Window of 4: max is positions 2-5: [2,10,23,3] = 38 OR [10,23,3,1] = 37 OR [23,3,1,0] = 27
# Actually: [10, 23, 3, 1] = 37? No -> [2, 10, 23, 3] = 38? Or further? Test carefully.
```

■ ***Tip:*** *Fixed-size window is simpler than variable-size. For variable, you adjust LEFT pointer based on a condition (like 'sum >= target'). Practice both.*

Onsite Coding

Q5. [Medium] Implement an LRU cache supporting get(key) and put(key, value), both O(1).

A: Doubly-linked list + hash map. The CANONICAL design-meets-coding question. Asked at Meta, Google, Amazon, etc.

```
class Node:
    def __init__(self, key=0, val=0):
        self.key = key
        self.val = val
        self.prev = None
        self.next = None

class LRUCache:
    """
    LRU = Least Recently Used.
    get and put both O(1).

    Data structure: doubly-linked list (for ordering) + hash map (for lookup).
    Most recently used = front of list. Least recently = back.

    Hash map: key -> Node
    Doubly-linked list: head (sentinel) <-> ... <-> tail (sentinel)
    """
    def __init__(self, capacity: int):
        self.cap = capacity
        self.cache = {}   # key -> Node
        # Sentinel nodes simplify edge cases
        self.head = Node()
        self.tail = Node()
        self.head.next = self.tail
        self.tail.prev = self.head

    def _remove(self, node: Node) -> None:
        """Unlink node from list."""
        node.prev.next = node.next
        node.next.prev = node.prev

    def _add_front(self, node: Node) -> None:
        """Insert node right after head (most-recent position)."""
        node.next = self.head.next
        node.prev = self.head
        self.head.next.prev = node
        self.head.next = node

    def get(self, key: int) -> int:
        if key not in self.cache:
            return -1
        node = self.cache[key]
        # Move to front (mark as most recent)
        self._remove(node)
        self._add_front(node)
        return node.val

    def put(self, key: int, value: int) -> None:
        if key in self.cache:
            # Update existing
            node = self.cache[key]
            node.val = value
            self._remove(node)
            self._add_front(node)
        else:
            # Insert new
            if len(self.cache) >= self.cap:
                # Evict LRU (right before tail sentinel)
                lru = self.tail.prev
                self._remove(lru)
                del self.cache[lru.key]
            new_node = Node(key, value)
            self.cache[key] = new_node
            self._add_front(new_node)

# Test
cache = LRUCache(2)
cache.put(1, 1)
cache.put(2, 2)
print(cache.get(1))   # 1
cache.put(3, 3)       # evicts 2
print(cache.get(2))   # -1 (gone)
cache.put(4, 4)       # evicts 1
print(cache.get(1))   # -1 (gone)
```

```
print(cache.get(3))   # 3
print(cache.get(4))   # 4

# Pythonic version: collections.OrderedDict
from collections import OrderedDict

class LRUCacheOrdered:
    def __init__(self, capacity: int):
        self.cap = capacity
        self.cache = OrderedDict()

    def get(self, key: int) -> int:
        if key not in self.cache:
            return -1
        self.cache.move_to_end(key)
        return self.cache[key]

    def put(self, key: int, value: int) -> None:
        if key in self.cache:
            self.cache.move_to_end(key)
        self.cache[key] = value
        if len(self.cache) > self.cap:
            self.cache.popitem(last=False)
```

■ ***Tip:*** *Mention OrderedDict as the 'Pythonic shortcut.' Most interviewers want the from-scratch version FIRST to verify you understand the data structures, then it's fine to show the shortcut.*

Onsite Coding

Q6. [Medium] Given a string s, find the length of the longest substring containing at most k distinct characters.

A: Variable-size sliding window with hash map. Very common pattern.

```
def longest_k_distinct(s: str, k: int) -> int:
    """
    Longest substring with at most k distinct characters.
    Variable sliding window.
    Time O(n), Space O(k).
    """
    if k == 0 or not s:
        return 0

    char_count = {}   # char -> count in current window
    left = 0
    best = 0

    for right in range(len(s)):
        char_count[s[right]] = char_count.get(s[right], 0) + 1

        # Shrink from left until at most k distinct
        while len(char_count) > k:
            char_count[s[left]] -= 1
            if char_count[s[left]] == 0:
                del char_count[s[left]]
            left += 1

        best = max(best, right - left + 1)

    return best

# Test
print(longest_k_distinct("eceba", 2))      # 3 ("ece")
print(longest_k_distinct("aa", 1))         # 2
print(longest_k_distinct("abcabcabc", 2))  # 2
```

■ ***Tip:*** *The 'shrink left while invalid' pattern works for many sliding-window problems. Internalize it.*

Onsite Coding

Q7. [Medium] Merge two sorted linked lists into one sorted list.

A: Two pointers + dummy node.

```
from typing import Optional
```

```
class ListNode:
    def __init__(self, val=0, next=None):
        self.val = val
        self.next = next

def merge_sorted(l1: Optional[ListNode], l2: Optional[ListNode]) -> Optional[ListNode]:
    """
    Merge two sorted lists.
    Time O(n + m), Space O(1) — reuses existing nodes.
    """
    dummy = ListNode()    # sentinel — avoids special-case for first append
    tail = dummy
    while l1 and l2:
        if l1.val <= l2.val:
            tail.next = l1
            l1 = l1.next
        else:
            tail.next = l2
            l2 = l2.next
        tail = tail.next
    # Attach whichever has remaining nodes
    tail.next = l1 if l1 else l2
    return dummy.next

# Recursive version (elegant but stack-heavy for long lists)
def merge_sorted_recursive(l1, l2):
    if not l1:
        return l2
    if not l2:
        return l1
    if l1.val <= l2.val:
        l1.next = merge_sorted_recursive(l1.next, l2)
        return l1
    else:
        l2.next = merge_sorted_recursive(l1, l2.next)
        return l2
```

■ ***Tip:*** *Dummy sentinel pattern is THE trick for linked-list construction. Memorize it.*

Onsite Coding

Q8. [Medium] Given an array of integers, return all subsets (the power set).

A: Backtracking. Foundational technique.

```
from typing import List

def subsets(nums: List[int]) -> List[List[int]]:
    """
    Return all 2^n subsets.
    Backtracking: at each element, choose include or exclude.
    Time O(n * 2^n), Space O(n * 2^n) for output.
    """
    result = []
    current = []

    def backtrack(start: int):
        # Every state of current is a valid subset
        result.append(current.copy())   # copy! current is mutated below
        for i in range(start, len(nums)):
            current.append(nums[i])
            backtrack(i + 1)
            current.pop()

    backtrack(0)
    return result

# Iterative version (bitmask):
def subsets_bitmask(nums: List[int]) -> List[List[int]]:
    n = len(nums)
    result = []
    for mask in range(1 << n):   # 0 to 2^n - 1
        subset = [nums[i] for i in range(n) if mask & (1 << i)]
        result.append(subset)
    return result
```

```
# Test
print(subsets([1, 2, 3]))
# [[], [1], [1,2], [1,2,3], [1,3], [2], [2,3], [3]]
```

■ ***Tip:*** *Backtracking template: choose → recurse → unchoose. Practice subsets, permutations, combinations — all variants of this pattern.*

Round 5: Onsite Coding Round 2 — Data Structures Deep-Dive (60 min)

Second onsite coding round, usually focused more on data structures and algorithms with conceptual depth. Expect: tree/graph problem + a discussion of underlying DS properties. The interviewer may pivot from coding to 'how does this work internally?' Be ready to talk about hash table internals, tree balancing, heap operations.

Onsite Coding

Q1. [Tree] Given a binary tree, return its inorder traversal — iterative, no recursion.

A: Explicit stack. Inorder = left, root, right.

```
from typing import Optional, List

class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val = val
        self.left = left
        self.right = right

def inorder_iterative(root: Optional[TreeNode]) -> List[int]:
    """
    Iterative inorder: left, root, right.
    Uses explicit stack instead of recursion.
    Time O(n). Space O(h) where h = tree height.
    """
    result = []
    stack = []
    current = root
    while current or stack:
        # Push all left children
        while current:
            stack.append(current)
            current = current.left
        # Pop, visit, move to right
        current = stack.pop()
        result.append(current.val)
        current = current.right
    return result

# Preorder iterative: just visit BEFORE going left
def preorder_iterative(root):
    if not root:
        return []
    result, stack = [], [root]
    while stack:
        node = stack.pop()
        result.append(node.val)
        # Push right first, so left pops next
        if node.right:
            stack.append(node.right)
        if node.left:
            stack.append(node.left)
    return result

# Postorder iterative: reverse of modified preorder
def postorder_iterative(root):
    if not root:
        return []
    result, stack = [], [root]
    while stack:
        node = stack.pop()
        result.append(node.val)
        if node.left:
            stack.append(node.left)
        if node.right:
```

```
            stack.append(node.right)
    return result[::-1]   # reverse
```

■ ***Tip:*** *Senior signal: 'I'd use iterative because Python's recursion limit (~1000) breaks on deep trees.' Real production knowledge.*

Onsite Coding

Q2. [Tree] Given a binary tree, find the lowest common ancestor of two nodes.

A: Classic recursive question.

```
from typing import Optional

class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val = val
        self.left = left
        self.right = right

def lowest_common_ancestor(root: TreeNode, p: TreeNode, q: TreeNode) -> TreeNode:
    """
    LCA of p and q in a binary tree.
    Recursive: if p or q found in subtree, return it. If both sides return non-null, current node is LCA.
    Time O(n), Space O(h) recursion.
    """
    if not root or root == p or root == q:
        return root

    left = lowest_common_ancestor(root.left, p, q)
    right = lowest_common_ancestor(root.right, p, q)

    if left and right:
        return root      # p and q split here — LCA is current node
    return left or right  # both on same side

# BST version is faster — use ordering:
def lca_bst(root, p, q):
    """
    For BST: if both p, q < root, go left; both > root, go right;
    else current is LCA.
    Time O(h).
    """
    while root:
        if p.val < root.val and q.val < root.val:
            root = root.left
        elif p.val > root.val and q.val > root.val:
            root = root.right
        else:
            return root   # split happens here
    return None
```

■ ***Tip:*** *Be ready for follow-up: 'What if it's a BST?' (use ordering for O(h)). 'What if you have parent pointers?' (two-pointer on ancestor chains).*

Onsite Coding

Q3. [Graph] Detect if a directed graph has a cycle.

A: DFS with three-color marking (white/gray/black). Back-edge = cycle.

```
from typing import Dict, List, Set
from enum import IntEnum

class Color(IntEnum):
    WHITE = 0   # unvisited
    GRAY = 1    # in current DFS path
    BLACK = 2   # fully processed

def has_cycle_directed(graph: Dict[int, List[int]]) -> bool:
    """
    Detect cycle in directed graph using DFS with 3 colors.
    Back-edge to a GRAY node = cycle.

    Time O(V + E). Space O(V).
    """
```

```
    color = {node: Color.WHITE for node in graph}

    def dfs(node: int) -> bool:
        color[node] = Color.GRAY
        for neighbor in graph.get(node, []):
            if neighbor not in color:
                color[neighbor] = Color.WHITE
            if color[neighbor] == Color.GRAY:
                return True   # back edge — cycle
            if color[neighbor] == Color.WHITE and dfs(neighbor):
                return True
        color[node] = Color.BLACK
        return False

    for node in graph:
        if color[node] == Color.WHITE:
            if dfs(node):
                return True
    return False

# Topological sort (Kahn's) — alternative cycle detection:
from collections import defaultdict, deque

def topological_sort(graph: Dict[int, List[int]]) -> List[int]:
    """
    Returns topological order. Raises if cycle exists.
    Used in: build dependencies, course prerequisites.
    Time O(V + E).
    """
    in_degree = defaultdict(int)
    for node in graph:
        in_degree[node]                 # ensure key exists
        for neighbor in graph[node]:
            in_degree[neighbor] += 1

    queue = deque([n for n in in_degree if in_degree[n] == 0])
    result = []
    while queue:
        node = queue.popleft()
        result.append(node)
        for neighbor in graph.get(node, []):
            in_degree[neighbor] -= 1
            if in_degree[neighbor] == 0:
                queue.append(neighbor)

    if len(result) != len(in_degree):
        raise ValueError("Cycle detected")
    return result

# For UNDIRECTED graphs, use Union-Find or DFS with parent tracking.
```

■ ***Tip:*** *Directed: 3-color or Kahn's. Undirected: Union-Find or parent-tracking DFS. Know the difference. Topological sort is itself a common interview question.*

Onsite Coding

Q4. [Graph] Number of islands — count connected components of 1s in a 2D grid (1=land, 0=water).

A: DFS or BFS from each unvisited 1.

```
from typing import List
from collections import deque

def num_islands(grid: List[List[str]]) -> int:
    """
    Count islands. Each island = connected component of '1's (4-directional).
    DFS approach.
    Time O(m*n), Space O(m*n) worst case.
    """
    if not grid or not grid[0]:
        return 0
    m, n = len(grid), len(grid[0])
    count = 0
```

```
    def dfs(r: int, c: int):
        if r < 0 or r >= m or c < 0 or c >= n or grid[r][c] != '1':
            return
        grid[r][c] = '0'   # mark visited (mutate grid)
        dfs(r + 1, c)
        dfs(r - 1, c)
        dfs(r, c + 1)
        dfs(r, c - 1)

    for r in range(m):
        for c in range(n):
            if grid[r][c] == '1':
                count += 1
                dfs(r, c)
    return count

# BFS version (preferred for very large grids to avoid recursion depth):
def num_islands_bfs(grid: List[List[str]]) -> int:
    if not grid or not grid[0]:
        return 0
    m, n = len(grid), len(grid[0])
    count = 0

    for r in range(m):
        for c in range(n):
            if grid[r][c] == '1':
                count += 1
                queue = deque([(r, c)])
                grid[r][c] = '0'
                while queue:
                    cr, cc = queue.popleft()
                    for dr, dc in [(-1, 0), (1, 0), (0, -1), (0, 1)]:
                        nr, nc = cr + dr, cc + dc
                        if (0 <= nr < m and 0 <= nc < n
                                and grid[nr][nc] == '1'):
                            grid[nr][nc] = '0'
                            queue.append((nr, nc))
    return count
```

■ ***Tip:*** *If interviewer asks 'don't modify the input grid' — use a separate visited set instead of mutating. O(m*n) space.*

Onsite Coding

Q5. [Heap] Given a stream of integers, design a class that returns the median at any time.

A: Two heaps: max-heap for lower half, min-heap for upper half.

```
import heapq
from typing import List

class MedianFinder:
    """
    Maintain median of stream in O(log n) per add, O(1) median.

    Data structure: two heaps.
    lower: max-heap (negated values) for smaller half.
    upper: min-heap for larger half.

    Invariants:
    - All in lower <= all in upper
    - |len(lower) - len(upper)| <= 1
    """
    def __init__(self):
        self.lower = []   # max-heap (use negation)
        self.upper = []   # min-heap

    def add_num(self, num: int) -> None:
        # Always push to lower first (negate for max-heap)
        heapq.heappush(self.lower, -num)
        # Move largest of lower to upper to maintain ordering
        heapq.heappush(self.upper, -heapq.heappop(self.lower))
        # Rebalance if sizes diverge
        if len(self.upper) > len(self.lower):
            heapq.heappush(self.lower, -heapq.heappop(self.upper))
```

```
    def find_median(self) -> float:
        if len(self.lower) > len(self.upper):
            return -self.lower[0]   # odd count, median is top of lower
        return (-self.lower[0] + self.upper[0]) / 2   # even, average

# Test
mf = MedianFinder()
mf.add_num(1)
mf.add_num(2)
print(mf.find_median())   # 1.5
mf.add_num(3)
print(mf.find_median())   # 2.0
```

■ ***Tip:*** *Two-heap pattern shows up everywhere. Variants: 'find median of sliding window' (harder — needs balanced BST for arbitrary delete).*

Conceptual

Q6. How does Python's dict work internally? Time complexity of get/put?

A: Hash table with open addressing + perturbation probing. Python 3.6+ split design: dense entries array + sparse hash index. (1) hash(key) computed via __hash__. (2) Modulo table size = initial slot. (3) If empty → miss. If occupied → check stored hash AND equality. If match → return. (4) If mismatch → 'perturb' to next slot using a sequence designed to avoid clustering. (5) Repeat until match or empty. Average O(1) for get/put. Worst case O(n) with adversarial hash collisions (Python adds hash randomization for str since 3.3 to defend against this). Resize: when load factor > 2/3, table doubles. Insertion order preserved as of 3.6 (was implementation detail, then language guarantee in 3.7). Memory: about half the size of pre-3.6 layout. Thread safety: GIL makes single bytecode operations atomic, but check-then-set still needs locks.

■ ***Tip:*** *Be ready to compare to Java HashMap (separate chaining, treeify on collision) or C++ unordered_map. Senior signal.*

Conceptual

Q7. What's the difference between a stack and a heap?

A: TWO meanings — clarify with interviewer. Data-structure stack vs memory-region stack. As DATA STRUCTURES: Stack is LIFO. Push and pop both O(1) at the top. Implementation: array or linked list. Used for: function call frames, parsing, DFS, undo. Heap is a tree satisfying heap property (min-heap: parent ≤ children). Implementation: array where parent at i, children at 2i+1, 2i+2. peek-min O(1), insert O(log n), extract-min O(log n), heapify-from-array O(n). Used for: priority queues, top-k, scheduling. Python's heapq is min-heap. As MEMORY REGIONS: Stack memory = thread-local, fixed size (usually 8MB), holds function frames (locals, return addresses). Grows down. Fast allocation (just decrement stack pointer). Limited size = recursion limit. Heap memory = process-wide, dynamically allocated via malloc/new. Holds long-lived objects. Slower allocation (alloc/free bookkeeping). Garbage-collected in managed languages.

■ ***Tip:*** *Always clarify which meaning. Senior signal: 'Data structure or memory region — which do you mean?'*

Conceptual

Q8. What's a hash collision and how does Python handle it?

A: Two different keys hashing to the same table slot. Without handling, second insert would overwrite first or be lost. Python uses OPEN ADDRESSING with PERTURBATION PROBING. On collision: compute a new probe sequence (not just linear +1, which would cluster) using a function of the hash and a perturbation value. Each probe lands in a different slot. Continue until empty slot found. Comparison: Java HashMap uses SEPARATE CHAINING — each slot is a linked list (or red-black tree after 8 collisions). C++ unordered_map uses separate chaining. Python uses open addressing because it's cache-friendlier for small dicts. Pathological case: adversarial keys with same hash → degenerates to O(n). Python defends with hash randomization for strings (PYTHONHASHSEED). The actual perturbation formula in CPython: `j = ((5*j) + 1 + perturb) % size; perturb >>= PERTURB_SHIFT` — produces good distribution.

■ ***Tip:*** *Mention that resize is triggered at load factor 2/3 — denser than typical 0.75, because open addressing degrades earlier.*

Conceptual

Q9. Explain quicksort. When does it perform poorly?

A: Divide-and-conquer sort. Pick a pivot, partition into [less] [equal] [greater], recursively sort the partitions. Average O(n log n) time, O(log n) recursion depth. Worst case O(n²) — when pivot is consistently the min or max (already-sorted input with first-element pivot). Mitigations: (1) random pivot — eliminates adversarial-input attacks. (2) Median-of-three pivot. (3) Introsort: switch to heapsort if recursion depth exceeds 2*log(n). (4) Three-way partition for arrays with many duplicates. Python's `sorted()` uses Timsort instead — O(n log n) WORST case, stable, exploits real-world partial sorting (already-sorted runs get O(n)). Quicksort is fast in practice due to cache-friendly access patterns, but Timsort wins on safety guarantees.

```
import random
from typing import List

def quicksort(arr: List[int]) -> List[int]:
    """Out-of-place quicksort. Random pivot avoids worst case."""
    if len(arr) <= 1:
        return arr
    pivot = random.choice(arr)
    less = [x for x in arr if x < pivot]
    equal = [x for x in arr if x == pivot]
    greater = [x for x in arr if x > pivot]
    return quicksort(less) + equal + quicksort(greater)
```

■ ***Tip:*** *Knowing 'Python uses Timsort, not quicksort' is a senior signal.*

Conceptual

Q10. What's a Trie and when would you use it?

A: Tree where each node represents a character; paths from root form strings. Each node has children indexed by character (or hash map for sparse). End-of-word flag marks complete words. Operations: insert O(L), search O(L), prefix search O(L) — all linear in string length, INDEPENDENT of how many words are stored. Use cases: autocomplete (find all words with given prefix), spell check (find similar words), longest common prefix, IP routing tables, regex compilation. Trade-off: memory-intensive when many unique characters. Compressed tries (Radix tree, Patricia trie) reduce this by merging chains.

```
class TrieNode:
    def __init__(self):
        self.children = {}
        self.is_word = False

class Trie:
    def __init__(self):
        self.root = TrieNode()

    def insert(self, word: str) -> None:
        node = self.root
        for c in word:
            if c not in node.children:
                node.children[c] = TrieNode()
            node = node.children[c]
        node.is_word = True

    def search(self, word: str) -> bool:
        node = self._find_node(word)
        return node is not None and node.is_word

    def starts_with(self, prefix: str) -> bool:
        return self._find_node(prefix) is not None

    def _find_node(self, s: str):
        node = self.root
        for c in s:
            if c not in node.children:
                return None
            node = node.children[c]
        return node
```

■ ***Tip:*** *Mention your Reddit search engine project — 'I used PyLucene which internally uses a similar structure for prefix lookups.' Connects to resume.*

Conceptual

Q11. Compare arrays and linked lists.

A: Arrays: contiguous memory. Random access O(1). Cache-friendly (sequential prefetching). Insert/delete in middle O(n) due to shifting. Resizing requires reallocation. Size known up front (or amortized O(1) for dynamic arrays). Linked lists: nodes scattered in memory, connected by pointers. Random access O(n). Cache-unfriendly (pointer chasing). Insert/delete given a node O(1). No size limit. Trade-offs: arrays for read-heavy or known-size data. Linked lists for frequent middle insertions. In practice, modern CPUs love sequential memory access — arrays win for traversal-heavy workloads. Linked lists shine in: LRU caches (combined with hash map), kernel data structures (task lists, free lists), buffer pools.

■ ***Tip:*** *Real production take: 'In Python, list is dynamic array (not actually linked). Use collections.deque for both-ends O(1) — it's a doubly-linked list of arrays.'*

Conceptual

Q12. Big-O complexities of common operations.

A: MEMORIZE THIS TABLE:

```
OPERATION COMPLEXITIES — interview cheat reference

ARRAYS (Python list):
  index access      O(1)
  append            O(1) amortized
  pop from end      O(1)
  insert at front   O(n)   — use deque if needed
  pop from front    O(n)   — use deque
  search by value   O(n)
  sort              O(n log n)

DICT (Python hash map):
  get/put/in        O(1) average, O(n) worst case
  iteration         O(n)

SET:
  add/remove/in     O(1) average
  union/intersect   O(|A| + |B|)

DEQUE (collections.deque):
  append/popleft    O(1)
  index access      O(n)

HEAP (heapq):
  push              O(log n)
  pop               O(log n)
  peek (h[0])       O(1)
  heapify           O(n)
  nlargest(k)       O(n log k)

LINKED LIST:
  insert at known   O(1)
  search by value   O(n)
  index access      O(n)

BST (balanced):
  insert/search/del O(log n)
  Python doesn't have built-in BST — use sortedcontainers.SortedList

GRAPH ALGORITHMS:
  BFS / DFS         O(V + E)
  Dijkstra (binary heap)  O((V + E) log V)
  Bellman-Ford      O(V * E)
  Floyd-Warshall    O(V^3)
  Topological sort  O(V + E)
  Union-Find        O(α(n)) per op (inverse Ackermann, nearly O(1))

SORTING:
  Quicksort avg     O(n log n), worst O(n^2)
  Mergesort         O(n log n) always
  Heapsort          O(n log n) always
```

```
  Timsort (Python)  O(n log n) worst, O(n) on nearly-sorted
  Counting sort     O(n + k) — when integers in known range
  Radix sort        O(d * n) — d = digits

STRING:
  contains substring         O(n) average (Python uses Boyer-Moore-like)
  startswith/endswith        O(k)
  split/join                 O(n)
```

■ ***Tip:*** *Memorize the table. Quote it confidently. 'Append is amortized O(1) — list doubles when full.' Senior signal.*

Round 6: System Design — Junior/NCG Edition (45-60 min)

For new grads, system design rounds are MORE FORGIVING than for senior. Interviewers don't expect you to design Twitter from scratch. They want to see: ability to clarify requirements, propose a structure, reason about trade-offs, listen to feedback. The discussion matters more than the final whiteboard. Use ARCH: Actors → Requirements → Components → High availability → Risks.

System Design

Q1. Design a URL shortener (like bit.ly).

A: Classic NCG question. Walk through structure.

```
# CLARIFYING QUESTIONS first:
# - Scale: how many URLs/day? (Assume 100M/day = ~1200/sec)
# - Read:write ratio? (Typical: 100:1 — most requests are redirects)
# - Custom slugs allowed?
# - Expiration / TTL?
# - Analytics needed?

# COMPONENTS:

User -> [Load Balancer] -> [App Servers] -> [Cache (Redis)] -> [DB]
                                  |
                                  v
                          [URL Shortener Service]

# Core operations:
# POST /shorten { "url": "https://very.long.com/path" }
#   -> Generate short code (8 chars: [a-zA-Z0-9])
#   -> Store mapping: short_code -> long_url
#   -> Return shortened URL

# GET /<short_code>
#   -> Look up long_url in cache; fallback to DB
#   -> HTTP 302 redirect to long_url
#   -> (Optional) log analytics event

# SHORT CODE GENERATION — three approaches:

# 1. Random — generate 8-char string, retry on collision
#    Pros: simple. Cons: collision check = extra read.

# 2. Counter-based — auto-incrementing ID, base62 encoded
#    Pros: no collisions. Cons: predictable (security concern).
#    Mitigate: encrypt or shuffle the counter.

# 3. Hash-based — MD5/SHA of URL, take first 8 chars
#    Pros: same URL gives same short code (dedup).
#    Cons: collisions possible — must check + adjust.

# DATA MODEL:
# Table: urls
#   id          BIGINT PRIMARY KEY AUTO_INCREMENT
#   short_code  VARCHAR(8) UNIQUE INDEX
#   long_url    TEXT NOT NULL
#   created_at  TIMESTAMP
#   expires_at  TIMESTAMP NULL
#   click_count BIGINT DEFAULT 0   -- updated async

# STORAGE:
# - Primary: relational DB (PostgreSQL/MySQL) for the mapping
# - Cache: Redis for read-heavy lookups (LRU eviction)
# - Analytics: append-only event log (Kafka -> data warehouse)

# SCALE LEVERS:
# - DB sharding by short_code prefix at 1B+ URLs
# - CDN at edge for popular URLs
# - Multi-region read replicas for global reads
# TRADE-OFFS:
```

```
# - Shorter codes = more potential collisions, but smaller URLs
# - Longer codes = more space, but cleaner
# - 8 chars * 62 options = 218 trillion combinations — plenty

# FAILURE MODES:
# - DB outage: cache absorbs reads for hot URLs; new shortens fail
# - Cache outage: traffic falls to DB, may overload — circuit breaker
# - Hot key (viral URL): CDN absorbs; cache layer holds
```

■ ***Tip:*** *ALWAYS ask clarifying questions FIRST. NCG who jumps into code without clarifying = red flag. Senior pattern.*

System Design

Q2. Design a rate limiter for an API.

A: Classic. Multiple algorithms — discuss trade-offs.

```
# REQUIREMENTS:
# - Limit per API key: e.g., 100 requests per minute
# - Distributed: works across multiple app servers
# - Low latency: < 5ms per check
# - Fail-open or fail-closed? (For most APIs: fail-open)

# ALGORITHMS:

# 1. FIXED WINDOW
# - For each (key, window_start), increment counter
# - Reject if > limit
# - Pros: simple. Cons: burst at window boundary (200 in 2 sec)

key = "user:123:202605161200"   # bucket by minute
count = redis.incr(key)
if count == 1:
    redis.expire(key, 60)
if count > 100:
    return reject()

# 2. SLIDING WINDOW LOG
# - Store timestamps of each request in a sorted set
# - Count timestamps within last minute
# - Pros: accurate. Cons: O(n) memory per key

key = f"user:123:log"
now = time.time()
redis.zremrangebyscore(key, 0, now - 60)   # remove old
count = redis.zcard(key)
if count >= 100:
    return reject()
redis.zadd(key, {now: now})
redis.expire(key, 60)

# 3. SLIDING WINDOW COUNTER
# - Combine two fixed windows with weighted estimation
# - Pros: smoother than fixed. Cons: approximate.

# 4. TOKEN BUCKET (most flexible)
# - Bucket holds N tokens, refills at rate R per second
# - Each request consumes 1 token; reject if empty
# - Allows bursts up to bucket size
# - Pros: handles bursts gracefully

# Implementation (in Redis):
class TokenBucket:
    def __init__(self, capacity: int, refill_rate: float):
        self.capacity = capacity
        self.refill_rate = refill_rate

    def allow(self, key: str) -> bool:
        now = time.time()
        # Atomic Lua script for consistency
        # Fetch current tokens + last refill time
        # Refill: tokens = min(capacity, tokens + elapsed * refill_rate)
        # If tokens >= 1: decrement and allow. Else reject.
        return True  # simplified
```

```
# STORAGE:
# - Redis cluster (shard by key hash)
# - In-memory per app server WITH async sync (eventual consistency)
# - DynamoDB Global Tables for cross-region

# FAILURE MODES:
# - Redis down: fall back to per-app local counter (degraded)
# - Clock skew: use Redis time, not client time
# - Hot key (viral user): consider local cache + Redis sync
# - Distributed inconsistency: budget for ~5% over-allowance
```

■ ***Tip:*** *Mention the four algorithms by name. Most candidates know one. Discussing trade-offs shows depth.*

System Design

Q3. Design a chat application like WhatsApp.

A: Broad question — keep it structured. Don't try to design every feature.

```
# CLARIFY first:
# - 1-on-1 only, or also groups?
# - Real-time delivery (websocket) or polling OK?
# - Read receipts? Typing indicators? Delivery confirmations?
# - End-to-end encryption?
# - Scale: assume 1M concurrent users to start

# HIGH-LEVEL ARCHITECTURE:

[Mobile/Web Client]
       |
       v
[Load Balancer] (sticky session for websocket)
       |
       v
[App Servers] <--> [Message Queue (Kafka)] <--> [Async workers]
       |                                              |
       v                                              v
[Cache (Redis)]                                [Notification service]
       |                                              |
       v                                              v
[Primary DB]                                   [Push: APNs, FCM]
(PostgreSQL or                                 [SMS gateway]
 Cassandra for messages)

# CORE FLOWS:

# 1. User comes online
# - Authenticates, opens WebSocket
# - Server stores: user_id -> connection_id, in Redis
# - Fetches missed messages since last_seen_at

# 2. User A sends to User B
# - A's app sends message over WebSocket to its app server
# - Server stores message in DB (message_id, from, to, body, ts)
# - Server checks if B online (Redis lookup)
#   - YES: push to B's WebSocket directly
#   - NO: queue notification (push notification via FCM/APNs)
# - Server sends ACK to A
# - Async: write to long-term log for analytics

# 3. User goes offline
# - Connection close event
# - Remove from connections map in Redis
# - Update last_seen_at

# DATA MODEL:
# users(id, name, last_seen_at, ...)
# messages(id, from_id, to_id, content, sent_at, delivered_at, read_at)
# group_members(group_id, user_id, joined_at)
# group_messages(id, group_id, from_id, content, sent_at)

# SCALE CONSIDERATIONS:
# - 1M concurrent: need many app servers, sticky load balancing
# - Messages table: shard by user_id; archive old messages
# - Cassandra is better for messages than PostgreSQL at scale
```

```
#   (wide-column, eventually consistent, write-optimized)
# - Group chats: fan-out at write (small groups) vs fan-out at read (large)

# RELIABILITY:
# - Message ordering: per-conversation sequence numbers
# - Delivery guarantee: at-least-once via ACK + retry; client dedups
# - Offline messages: store in DB until read or TTL

# E2E ENCRYPTION (Signal Protocol):
# - Each user has identity key + pre-keys
# - First message establishes session via X3DH
# - Subsequent messages: Double Ratchet algorithm
# - Server stores ciphertext only — can't read

# FAILURE MODES:
# - WebSocket drops: client reconnects with last received message ID
# - Server failure: load balancer reroutes; client gets ack from new server
# - Message storm (popular group): rate limit at app layer
```

■ ***Tip:*** *Don't over-engineer. Focus on the 3 core flows. Mention E2E as 'I'd add Signal Protocol' but don't try to whiteboard it unless asked.*

System Design

Q4. Design a movie recommendation system.

A: Tie to YOUR anime recommender project — ALS + KMeans!

```
# YOUR project: Spark MLlib ALS + KMeans on 17K-title dataset.
# Frame this as: 'I built a small version of this at school using Spark MLlib.
# Here's how I'd scale it to Netflix-style:'

# REQUIREMENTS:
# - Personalized recommendations per user
# - Cold start: new users and new items
# - Real-time: update with recent interactions
# - Scale: 100M users, 10K items, billions of interactions

# COMPONENTS:

[User behavior events] -> [Kafka] -> [Real-time processor]
                                            |
                                            v
                                    [Feature store]
                                            |
                                            v
[Offline training pipeline]      [Online inference service]
       |                                    |
       v                                    v
   [Model registry] <----- pulls --- [Recommendation API]
       |
       |
       v
[Batch precomputed recs] -> [Cache]

# THREE RECOMMENDATION APPROACHES (use blend):

# 1. COLLABORATIVE FILTERING (what your ALS project did)
#    - Matrix factorization: U * V^T ~= R (rating matrix)
#    - Learn user embeddings and item embeddings
#    - Score(u, i) = u_embedding . i_embedding
#    - Spark MLlib's ALS does this. Output: top-k for each user.
#    - WEAKNESS: cold start. New user/item has no embeddings.

# 2. CONTENT-BASED
#    - Item features: genre, cast, director, year, language, plot embedding
#    - User profile = avg of features of items they liked
#    - Score by cosine similarity
#    - HANDLES COLD START for items: new movie has features immediately.

# 3. DEEP LEARNING / NEURAL CF
#    - Two-tower model: user tower + item tower
#    - Each tower learns embedding from features
#    - Score by dot product
#    - Production: TensorFlow / PyTorch, served via Triton
```

```
# HYBRID = sum or learned blend of all three signals.

# RANKING PIPELINE:
# Stage 1 (Candidate generation): get top 1000 candidates
#   - From CF (your strength)
#   - From content-based for new users/items
#   - From popular-in-region
#
# Stage 2 (Ranking): score candidates with rich model
#   - Features: user history, time of day, device, recent watches
#   - Output: top 50 ranked recommendations
#
# Stage 3 (Re-ranking / business rules):
#   - Diversity (don't all be sci-fi)
#   - Promote new content
#   - Exclude already-watched

# FEEDBACK LOOP:
# - User clicks/watches/skips -> Kafka event
# - Train model on (features, label = watched/skipped)
# - A/B test new models before full rollout

# COLD START SOLUTIONS:
# - New user: ask onboarding preferences ('what 5 movies do you like?')
# - New item: use content features only initially; learn embedding over weeks
# - Fallback: most-popular-in-segment

# YOUR resume connection:
# 'In my anime project, I used ALS + KMeans on 17K titles. KMeans clustered users
#  by behavior pattern. ALS learned user and item embeddings. The Flask + React
#  frontend let users explore recommendations interactively. To scale this to
#  Netflix's level, I'd add streaming feature updates via Kafka and a two-tower
#  deep model for ranking on top of ALS candidates.'
```

■ ***Tip:*** *TIE THIS DIRECTLY TO YOUR ANIME RECOMMENDER. 'I built a small version' immediately establishes credibility on this question.*

System Design

Q5. Design a search engine for Reddit-style content.

A: Your Reddit search engine project — own this answer.

```
# YOUR project: PyLucene + BM25 + Flask. Scale up the discussion.

# REQUIREMENTS:
# - Search across millions of posts and comments
# - Fast results (< 200ms)
# - Relevance ranking
# - Real-time indexing (new posts searchable within seconds)
# - Faceted filters: subreddit, time, score, author

# ARCHITECTURE:

[Reddit content] -> [Kafka] -> [Indexer service] -> [Search index]
                                                        ^
                                                        |
[User search] -> [API] -> [Query parser] -> [Index query]
                                                        |
                                                        v
                                                  [Ranker] -> Results

# CORE COMPONENTS:

# 1. INDEX (Apache Lucene / Elasticsearch / Solr)
# - Inverted index: token -> list of (doc_id, position, frequency)
# - Posting lists compressed (delta encoding + variable-byte)
# - Sharded by hash of post_id for parallelism
# - Replicas for read scaling

# 2. TOKENIZATION + ANALYSIS
# - Lowercase, stop word removal
# - Stemming (running -> run)
# - N-grams for fuzzy match (typos)
# - Language detection per document
```

```
# Your PyLucene project: this is exactly what Lucene's analyzers do

# 3. SCORING (BM25 — what you used!)
# - BM25(q, d) = sum over terms of: idf(t) * tf_normalized(t, d)
# - idf rewards rare terms (good for distinguishing)
# - tf_normalized handles document length normalization
# - Tuneable params: k1 (term saturation), b (length norm)

# 4. ADDITIONAL SIGNALS:
# - Recency (newer posts ranked higher)
# - Authority (subreddit/user reputation)
# - Engagement (upvotes, comment count)
# - Personalization (user's subreddit preferences)
# Final: blended score = w1*BM25 + w2*recency + w3*authority + ...

# 5. QUERY PROCESSING:
# - Parse user query into tokens
# - Handle phrase queries ("exact match")
# - Boolean operators (term1 AND term2)
# - Auto-suggest as user types (typeahead)

# SCALE:
# - Horizontal: more shards
# - Vertical: faster machines for hot indexes
# - Cache hot queries in Redis
# - CDN for static content

# YOUR resume connection:
# 'In my Reddit search engine, I used PyLucene with BM25 ranking and built a Flask
#  front-end. I'd extend this to production by adding: distributed Elasticsearch
#  for sharding, real-time indexing via Kafka, learning-to-rank for relevance,
#  and a typeahead service for query suggestion. The core BM25 formula stays —
#  it's still the best single-stage ranker. ML adds reranking on top.'
```

■ ***Tip:*** *Pull from PyLucene knowledge — it's a real superpower for search-design questions.*

System Design

Q6. Design a system to detect anomalies in a stream of sensor data (or fraud detection).

A: Real-time ML pipeline.

```
# REQUIREMENTS:
# - Sensor / transaction data: thousands of events per second
# - Detect anomalies in real time (< 1 sec)
# - Reasons: fraud (financial), failure prediction (industrial), security (network)

# ARCHITECTURE:

[Sensors / Events] -> [Kafka] -> [Stream processor (Flink/Spark)]
                                          |
                                          v
                                  [Feature engineering]
                                          |
                                          v
                                  [ML model inference]
                                          |
                                  Score < threshold? -> ALERT
                                          |
                                          v
                                  [Decision store]
                                          |
                                          v
                                  [Action: block / flag / notify]

# FEATURE ENGINEERING (online):
# - Time-windowed stats: avg, std, percentile over last 5 min
# - Rate of change
# - Outlier flags from rolling z-scores
# - Cross-feature correlations
# - Pre-computed user/account-level stats from feature store

# MODELS (typically blend):
# - Statistical: z-score, EWMA control charts (cheap baseline)
```

```
# - Isolation Forest (unsupervised, no labeled fraud needed)
# - Autoencoders (reconstruct normal; high reconstruction error = anomaly)
# - Supervised: XGBoost / Random Forest on labeled fraud (when available)

# REAL-TIME INFERENCE:
# - Model deployed as low-latency service (Triton, BentoML, TF Serving)
# - Each event scored in < 50ms
# - Threshold tuned per use case: high recall (catch all fraud) vs precision

# FEEDBACK LOOP:
# - Confirmed fraud -> labeled positive
# - Retrain model weekly with new labels
# - A/B test new model vs incumbent before full rollout

# CHALLENGES:
# - Label scarcity: confirmed fraud is rare
#   Solution: semi-supervised learning, anomaly detection
# - Concept drift: fraud patterns change
#   Solution: continuous monitoring of model performance
# - Adversarial: fraudsters adapt
#   Solution: regular model retraining, ensemble of models
# - False positives: bad UX (blocked legit user)
#   Solution: tunable thresholds, escalation to manual review

# YOUR resume connection:
# 'My KNN feature selection project was small-scale anomaly detection — picking
#  which features distinguish classes. For production, I'd use Isolation Forest
#  for unsupervised detection (no labels needed) and XGBoost on confirmed cases.
#  The Reddit search engine taught me Kafka-style streaming — same architecture.'
```

■ ***Tip:*** *Bring up your KNN feature selection and Spark MLlib work. Real student projects map well to anomaly detection.*

Round 7: AI/ML Domain Round (45-60 min)

Increasingly common at NVIDIA, Google, Meta, Apple, NVIDIA, Microsoft — even for non-research roles. Tests: ML fundamentals, NLP basics, applied judgment. New grad bar: solid understanding of basic algorithms (linear regression, decision trees, KNN, neural net basics, training/inference), familiarity with common frameworks (PyTorch, TF, scikit-learn, transformers). Your project background — NLP TextBlob, Spark MLlib ALS/KMeans, scikit-learn at Delta — is well-aligned.

AI/ML

Q1. Walk me through how a neural network learns.

A: Forward pass + backprop + optimizer. The core loop.

```
# Step-by-step in plain words:

# 1. FORWARD PASS
#    Input flows through network layers.
#    Each layer: output = activation(W * input + b)
#    W and b are learnable parameters.
#    Final layer produces prediction.

# 2. LOSS COMPUTATION
#    Compare prediction to true label.
#    Common losses:
#    - Classification: cross-entropy
#    - Regression: MSE (mean squared error)
#    - Embedding: contrastive / triplet loss

# 3. BACKPROPAGATION
#    Compute gradient of loss w.r.t. each parameter (W, b).
#    Chain rule applied layer-by-layer, back to front.
#    Result: dL/dW for every parameter.

# 4. OPTIMIZER STEP
#    Update parameters: W = W - learning_rate * dL/dW
#    Variants:
#    - SGD: plain stochastic gradient descent
#    - SGD + momentum: smooths updates with running velocity
#    - Adam: adaptive learning rate per parameter (most popular)
#    - AdamW: Adam with decoupled weight decay

# 5. REPEAT for many epochs over data, with mini-batches.

# Code (PyTorch):
import torch
import torch.nn as nn
import torch.optim as optim

model = nn.Sequential(
    nn.Linear(784, 256), nn.ReLU(),
    nn.Linear(256, 10)
)
optimizer = optim.Adam(model.parameters(), lr=1e-3)
loss_fn = nn.CrossEntropyLoss()

for epoch in range(10):
    for batch_x, batch_y in train_loader:
        optimizer.zero_grad()              # reset gradients
        logits = model(batch_x)             # forward
        loss = loss_fn(logits, batch_y)     # compute loss
        loss.backward()                     # backprop
        optimizer.step()                    # update weights
```

■ ***Tip:*** *Be ready for follow-ups: 'What's the vanishing gradient problem?' (sigmoid/tanh saturate at extremes — use ReLU). 'What's batch normalization?' (normalize layer inputs to stabilize training).*

AI/ML

Q2. What's overfitting? How do you prevent it?

A: Model memorizes training data, fails to generalize. Train loss low, validation loss high.

```
# DIAGNOSING OVERFITTING:
# - Train accuracy >> validation accuracy
# - Validation loss starts decreasing then INCREASES (model gets worse)
# - Model performs well on seen data, poorly on new

# PREVENTION TECHNIQUES:

# 1. MORE DATA (best fix)
# - Collect more, or use data augmentation
# - For images: rotate, flip, crop, color jitter
# - For text: synonyms, back-translation, paraphrase

# 2. REGULARIZATION
# - L2 (weight decay): penalize large weights
#   loss = original_loss + lambda * sum(W^2)
# - L1: sparse weights (zero out unimportant features)
# - Dropout: randomly zero out neurons during training
#   nn.Dropout(p=0.5)

# 3. EARLY STOPPING
# - Monitor validation loss
# - Stop when it starts increasing
# - Save the best checkpoint

# 4. SIMPLER MODEL
# - Fewer parameters
# - Shallower network
# - Smaller embedding dimension

# 5. CROSS-VALIDATION
# - K-fold for small datasets
# - Stratified split for imbalanced classes

# 6. ENSEMBLE
# - Average predictions from multiple models
# - Different architectures, different random seeds
# - Bagging, boosting

# YOUR resume connection:
# 'In my KNN feature selection project, I used forward + backward selection
#  to reduce features and prevent overfitting. Validation-driven selection
#  was the key — pick the feature set that maximized HELD-OUT accuracy.'
```

■ ***Tip:*** *Forward/backward selection from your KNN project IS regularization at the feature level. Mention this.*

AI/ML

Q3. What's the difference between supervised, unsupervised, and reinforcement learning?

A: Foundational. Be crisp.

```
# SUPERVISED LEARNING
# - Have labeled examples: (input X, label y) pairs
# - Goal: predict y for new X
# - Examples: classification, regression
# - Algorithms: linear regression, logistic regression, SVM, neural nets,
#   random forest, XGBoost, KNN

# UNSUPERVISED LEARNING
# - Have only inputs X, no labels
# - Goal: find structure in data
# - Examples:
#   - Clustering (group similar): KMeans, DBSCAN, hierarchical
#   - Dimensionality reduction: PCA, t-SNE, UMAP, autoencoders
#   - Anomaly detection: isolation forest, one-class SVM
#   - Association: Apriori, market basket

# REINFORCEMENT LEARNING
# - Agent interacts with environment, gets rewards
# - Goal: maximize cumulative reward over time
# - State -> Action -> Reward + Next State
# - Examples: game playing (AlphaGo), robotics, recommender systems
```

```
# - Algorithms: Q-learning, DQN, policy gradient, PPO, A2C

# SEMI-SUPERVISED (the hybrid):
# - Few labeled examples + many unlabeled
# - Use unlabeled data to learn structure, then refine with labeled
# - Common in real production where labeling is expensive

# YOUR resume connection:
# 'My anime recommender used Spark MLlib ALS — a form of collaborative filtering
#  that learns embeddings from implicit feedback (user behavior, not explicit
#  ratings). It's technically self-supervised — the labels come from the data itself.
#  The KMeans clustering on top was pure unsupervised — find user segments.'
```

■ ***Tip:*** *Mention SELF-SUPERVISED briefly — that's how modern LLMs train (next-word prediction is supervised but labels come from raw data).*

AI/ML

Q4. How does a Large Language Model (like GPT or Llama) work at a high level?

A: Token-by-token autoregressive prediction using transformer architecture.

```
# HIGH-LEVEL:

# 1. TOKENIZATION
# - Convert text to tokens (sub-word pieces using BPE or SentencePiece)
# - Each token has an integer ID
# - 'Hello, world!' -> [9906, 11, 1917, 0]

# 2. EMBEDDING
# - Each token ID maps to a vector (e.g., 4096-dim)
# - Learned during training

# 3. TRANSFORMER LAYERS (e.g., 32-80 layers)
# - Each layer has:
#   - SELF-ATTENTION: each token attends to all others (weighted by similarity)
#   - FEED-FORWARD: nonlinear transformation per token
#   - Residual connections + layer norm

# 4. OUTPUT
# - Final layer projects back to vocabulary size
# - Softmax -> probability over next token

# 5. AUTOREGRESSIVE GENERATION
# - Predict next token, append to input
# - Repeat
# - Sampling strategies:
#   - Greedy (always pick max prob — deterministic, often boring)
#   - Top-k (sample from top k tokens)
#   - Top-p / nucleus (sample from smallest set with cumulative prob >= p)
#   - Temperature (sharpen or flatten distribution)

# TRAINING:
# - Pre-training: predict next token on huge text corpus
# - Fine-tuning: supervised on task-specific data
# - RLHF: refine with human preferences

# KEY INNOVATIONS:
# - Attention mechanism (Vaswani 2017)
# - Scaling laws (more params + more data = better)
# - In-context learning (prompt as task spec)
# - Tool use / function calling (newer models)

# YOUR resume connection:
# 'My TextBlob NLP feature at LifeStages used pre-trained sentiment models.
#  Modern LLMs do this and much more in zero-shot — give them an instruction,
#  they execute. The progression from TextBlob -> BERT -> GPT is one of the
#  most exciting story arcs in CS. I want to work on whatever's next.'
```

■ ***Tip:*** *DON'T pretend to know transformer math in detail unless you do. Stay high-level. Mention TextBlob -> modern LLM arc to connect to your resume.*

AI/ML

Q5. Explain how convolutional neural networks (CNNs) work.

A: Image-specific architecture exploiting spatial locality.

```
# WHY CNNs FOR IMAGES?
# - Fully-connected (MLP) on 224x224x3 image: 150K input -> millions of weights
# - Doesn't exploit that nearby pixels are related
# - CNN reuses small filters across whole image -> fewer parameters

# CORE OPERATIONS:

# 1. CONVOLUTION
# - Filter (e.g., 3x3) slides over image
# - At each position, compute dot product
# - Output: feature map highlighting that pattern
# - Many filters -> many feature maps -> stacked output

# 2. ACTIVATION (ReLU)
# - Add non-linearity
# - max(0, x)

# 3. POOLING (max or average)
# - Downsample: take max of each 2x2 window
# - Reduces spatial size, adds translation invariance

# 4. STACK
# - Early layers: detect edges, textures
# - Middle layers: detect parts (eyes, wheels)
# - Late layers: detect objects (faces, cars)
# - Final fully-connected: classify

# FAMOUS ARCHITECTURES:
# - LeNet (1998): first practical CNN
# - AlexNet (2012): ignited deep learning revolution on ImageNet
# - VGG: deeper, simpler (just 3x3 convs)
# - ResNet: residual connections enable 100+ layers
# - EfficientNet: balanced depth/width/resolution
# - Vision Transformers (ViT): newer alternative using attention instead

# YOUR resume connection:
# 'My CUDA work made me think about why CNNs are so GPU-friendly: convolutions
#  are highly parallel — each output pixel can be computed independently.
#  That's why GPUs (and TPUs, and NVIDIA Tensor Cores) are designed for this.'
```

■ ***Tip:*** *If asked details: mention Tensor Cores (NVIDIA-specific). Connects to NVIDIA hardware story.*

AI/ML

Q6. What's the bias-variance tradeoff?

A: Foundational ML concept. Be crisp.

```
# BIAS = systematic error from oversimplifying.
# - Model can't capture the true pattern.
# - Example: linear model on quadratic data.
# - Symptom: underfitting. Train AND validation both bad.

# VARIANCE = error from sensitivity to specific training data.
# - Model captures noise as if it were signal.
# - Example: 10-degree polynomial on linear data.
# - Symptom: overfitting. Train good, validation bad.

# TRADEOFF:
# - Increase model complexity -> bias down, variance up
# - Simpler model -> higher bias, lower variance
# - Goal: minimize total error = bias^2 + variance + irreducible_error

# HOW TO TUNE:
# - Underfitting (high bias): make model bigger, train longer, add features
# - Overfitting (high variance): regularize, get more data, simpler model

# MODERN VIEW:
# - With deep learning, more parameters can sometimes REDUCE variance
#   (the "double descent" phenomenon)
# - Modern LLMs have billions of parameters AND generalize well
# - Theory still being developed
```

■ ***Tip:*** *Mention double descent as a senior signal — most NCG candidates have never heard of it.*

AI/ML

Q7. Walk me through your anime recommendation project.

A: Be ready to go deep — this is YOUR resume.

```
# Project context (yours):
# - 17K-title dataset
# - Spark MLlib ALS + KMeans
# - Flask + React frontend
# - SQLite backend

# YOUR PIPELINE:

# 1. DATA
# - Anime metadata: title, genre, episodes, score, members
# - User ratings (implicit + explicit)
# - Cleaned and joined in Spark

# 2. ALS — ALTERNATING LEAST SQUARES
# - Matrix factorization: R (m x n) ~= U (m x k) * V^T (k x n)
# - U = user embeddings, V = item embeddings, k = latent dim
# - Why ALS not SGD? ALS parallelizes well in Spark
# - Hyperparameters: rank (k), regParam (regularization), iterations

# Code outline:
from pyspark.ml.recommendation import ALS
als = ALS(
    userCol="userId", itemCol="animeId", ratingCol="rating",
    rank=10, regParam=0.1, maxIter=10,
    coldStartStrategy="drop"   # drops users with no overlap
)
model = als.fit(training_df)
recs = model.recommendForAllUsers(numItems=10)

# 3. KMEANS USER CLUSTERING
# - Cluster users by their ALS embeddings
# - Each cluster represents a "viewing pattern"
# - Useful for marketing or cold-start fallback

from pyspark.ml.clustering import KMeans
kmeans = KMeans(k=8, featuresCol="user_embedding")
cluster_model = kmeans.fit(user_features_df)

# 4. SERVING
# - Precompute top-N per user, store in SQLite
# - Flask API: GET /user/<id>/recs
# - React frontend with custom HTML/CSS layouts

# CHALLENGES YOU SOLVED:
# - Cold start: new users -> fallback to popular-in-cluster
# - Sparsity: most users only rate few items -> implicit feedback helps
# - Diversity: pure ALS top-10 is often all same genre -> re-rank for diversity

# WHAT YOU'D IMPROVE WITH MORE TIME:
# - Add content features (genre, year) as side info
# - A/B testing framework
# - Real-time updates via streaming
# - Move from SQLite to a real DB at scale
```

■ ***Tip:*** *If they push: 'Why ALS over SVD?' (handles missing entries naturally, parallelizes). 'Why k=10 latent dim?' (validation-tuned, balance overfitting). 'How would you A/B test?' (split users, compare CTR/dwell).*

AI/ML

Q8. What's transfer learning?

A: Reuse a model trained on one task for a different (related) task.

```
# WHY: training from scratch needs lots of data + compute.
# Transfer learning: take a model trained on a large dataset (e.g., ImageNet),
# fine-tune on your smaller dataset.
# COMMON PATTERN:
```

```
# 1. Take pre-trained model (e.g., ResNet on ImageNet, BERT on text)
# 2. Replace final layer with new layer for YOUR task
# 3. Freeze early layers (general features) OR fine-tune everything
# 4. Train on your data — fast because most weights are already good

# DECISION TREE:
# - Lots of data, similar to source domain: fine-tune all layers
# - Lots of data, different domain: train more layers from scratch
# - Little data, similar domain: freeze most, train just classifier
# - Little data, different domain: hard problem — try data augmentation,
#   few-shot learning, or different base model

# Examples:
# - BERT -> sentiment classifier (huge text -> small label set)
# - ResNet -> medical imaging (millions of natural images -> X-rays)
# - GPT -> domain expert (general text -> code, legal, medical)

# YOUR resume connection:
# 'My TextBlob NLP feature was a form of transfer learning — TextBlob ships
#  with pre-trained models for sentiment and noun phrase extraction. I used
#  those out of the box for emotional tone categorization without training
#  anything myself. For a senior version, I'd fine-tune a BERT classifier on
#  labeled mental wellness journal entries — that would adapt to the domain.'
```

■ ***Tip:*** *TextBlob is a perfect transfer learning example. Tie it explicitly.*

AI/ML

Q9. Difference between precision, recall, F1, ROC-AUC.

A: Classification metrics — must know cold.

```
# CONFUSION MATRIX:
#                   Predicted
#                Positive  Negative
# Actual Pos      TP        FN
# Actual Neg      FP        TN

# PRECISION = TP / (TP + FP)
# - Of all positive predictions, how many were correct?
# - 'When the model says yes, is it right?'
# - High precision = few false positives.

# RECALL = TP / (TP + FN)
# - Of all actual positives, how many did we catch?
# - 'When something IS positive, do we find it?'
# - High recall = few false negatives.

# F1 = 2 * (P * R) / (P + R)
# - Harmonic mean of precision and recall
# - Penalizes if either is low
# - Single number summary, but assumes equal importance

# WHEN TO PREFER:
# - Spam detection: precision matters (don't mark real email as spam)
# - Cancer screening: recall matters (don't miss a real case)
# - Imbalanced classes (fraud=1%, normal=99%): accuracy useless, use F1

# ROC CURVE:
# - Plot True Positive Rate vs False Positive Rate at various thresholds
# - AUC = Area Under Curve. 1.0 = perfect, 0.5 = random.
# - Good when class balance is similar.

# PRECISION-RECALL CURVE:
# - Plot precision vs recall at various thresholds
# - AUC-PR. Better than ROC-AUC for IMBALANCED classes.

# YOUR resume connection:
# 'In my KNN feature selection, validation accuracy was the proxy. For
#  imbalanced classes I'd switch to F1 or AUC-PR. For my Reddit search engine,
#  ranking metrics like NDCG matter more than classification metrics —
#  position of relevant results matters, not just whether they appear.'
```

■ ***Tip:*** *Mention NDCG (normalized discounted cumulative gain) for ranking — connects to your Reddit search project. Senior signal.*

AI/ML

Q10. Have you used LLMs / ChatGPT / Claude in your work?

A: Recent question — increasingly asked.

Be honest about real usage. Sample: 'Yes, daily. I use ChatGPT and Claude for: (1) Brainstorming approaches to a problem — especially when I'm stuck. (2) Code review on my own code — feed it the function, ask for issues. (3) Learning new APIs or libraries — faster than reading full docs. (4) Writing tests — describe the function, ask for edge cases. (5) Debugging stack traces. I'm careful about: (1) Not pasting proprietary code into public LLMs — at Delta I only use approved tools. (2) Verifying generated code — LLMs hallucinate APIs and miss edge cases. (3) Understanding what the LLM wrote — I don't ship code I can't explain. The skill I'm building: knowing when an LLM is a force multiplier vs when it's a shortcut to bad code. For most problems it's the former, but you have to keep your judgment.'

■ ***Tip:*** *Honesty + boundaries + judgment = senior signal. Pretending you don't use LLMs sounds outdated. Pretending you only use them = lazy.*

Round 8: Behavioral Round (45 min)

Tests culture fit, growth potential, communication. For NCG: lean heavily on COURSEWORK, INTERNSHIPS, and PROJECTS as your stories — you don't have years of corporate war stories yet, and that's OK. Use STAR rigorously. Have 5-7 polished stories ready that you can adapt to many questions. Companies that care most about behavioral: Amazon (Leadership Principles), Meta (move-fast culture), Google (Googleyness), Microsoft (growth mindset), Apple (intensity + craftsmanship), Netflix (judgment, candor).

Behavioral

Q1. Tell me about a time you had to learn something quickly.

A: Frame around your CUDA project or LangChain ramp-up. Sample: 'When I took GPU Programming, I had two weeks to learn CUDA well enough to deliver the BFS maze solver. I'd never written kernel code. My approach: I broke it down. Day 1-2: read Mark Harris's tutorials on parallel BFS to get the algorithm. Day 3-4: wrote a naive single-threaded version on GPU to learn the syntax. Day 5-7: parallelized the frontier expansion. Day 8-10: profiled with NVIDIA Nsight to find the bottleneck — uncoalesced memory access. Day 11-13: restructured the data layout and got the 12x speedup. Day 14: wrote the report. Lesson: when learning fast, build SOMETHING in the first 48 hours. The hands-on debugging surfaces gaps that hours of reading wouldn't.'

■ ***Tip:*** *Pick a REAL story. Generic 'I read a book' answers fail. The interviewer wants to see your learning process under constraint.*

Behavioral

Q2. Tell me about a time you worked on a team.

A: LifeStages team project. Sample: 'At LifeStages, I worked with three other interns on Align, the mental wellness app. I owned the NLP journaling feature, but it had to integrate cleanly with the Flutter frontend (owned by another intern) and Flask backend (a third). My approach: I scoped my contract first — what data my API would return, what format, what error cases. Shared this in a doc, got buy-in from the frontend and backend owners. Then I built and tested independently. Integration day was smooth because the contracts were stable. We shipped on time. Lesson: in any team project, define the interface FIRST. Saves a week of 'wait, your function returns what?' later.'

■ ***Tip:*** *Story should show interface-first thinking. That's the senior signal.*

Behavioral

Q3. Tell me about a time you disagreed with someone.

A: Show respect, data, and resolution. Sample: 'In my Delta co-op, my mentor wanted to parse the LEAP BoM XML with regex. I'd started prototyping with BeautifulSoup and felt strongly that XML parsing should use a proper parser — regex is famously bad at nested structures. I didn't just say 'you're wrong.' I built a small benchmark: same input, two implementations, compared correctness and maintainability. The BeautifulSoup version caught five edge cases the regex missed. I shared the benchmark with my mentor. He looked at it, agreed, and switched approach. Lesson: when disagreeing, bring DATA, not opinions. Don't make it personal — show the work.'

■ ***Tip:*** *Disagreement stories should ALWAYS end with respect intact. Never bash anyone.*

Behavioral

Q4. Tell me about a project you're most proud of.

A: Pick the CUDA project — deepest technical story. Sample: 'My CUDA BFS maze solver is what I'm most proud of, less for the 12x speedup than for what I learned along the way. The first version was 3x faster than CPU — fine, but not great. I instrumented with NVIDIA Nsight and found I was memory-bound from uncoalesced access. Restructured to a flat row-major grid and got to 7x. Then I noticed early BFS levels had tiny frontiers underutilizing the GPU. Added a fallback to CPU for sparse levels and got to 12x. What I'm proud of is the process: measure, hypothesize, fix, measure again. That's the engineering muscle I want to keep building.'

■ ***Tip:*** *Process over outcome. Senior signal.*

Behavioral

Q5. What's your biggest weakness?

A: Real weakness + how you manage it. Sample: 'I tend to over-perfect early in a project — spending too much time on edge cases before getting the basic version working end-to-end. My CUDA project taught me this — I spent 3 days on the first kernel before I had ANYTHING running. Now I force myself to ship a crap version first, then iterate. The hardest part of the project was getting ANY GPU code running; the optimization was the fun part. I'd rather have an ugly working pipeline by day 3 and pretty it up than have a beautiful design doc and no code.'

■ ***Tip:*** *Generic 'I work too hard' = lazy answer. Specific story + how you're actively addressing it = real.*

Behavioral

Q6. Tell me about a time you failed.

A: Show vulnerability + lesson. Sample: 'In my first month at Delta, I built an automated report that summarized part status across fleets. I deployed it for the team to use. After two days, my mentor caught that my filter logic was excluding some parts incorrectly — about 5% of the data was hidden. The team had been using the report and might have made decisions on it. I felt terrible. I rolled back the deploy, fixed the bug, wrote test cases for the filter, and ran the old reports for everyone who had used it to flag what they might have missed. Lesson: validate against ground truth before you ship. Even a 95% accurate report can mislead users who trust it. I now write data-quality assertions BEFORE I write the report code.'

■ ***Tip:*** *Best failure stories: real, fixable, lesson learned, behavior changed. Don't pick a failure that suggests you're unreliable.*

Behavioral

Q7. Tell me about a time you had to give difficult feedback.

A: Group project from school works well. Sample: 'In my AI class, I worked with three other students on the 8-Puzzle solver project. One teammate consistently committed broken code right before the deadline, breaking the build for everyone. I'd been silently fixing it. Two weeks before the final, I asked to talk with them privately. I said something like: I need your help, when you commit at midnight on the day before review I have to debug your code instead of working on my section, can we agree to merge by 6pm with passing tests? They were defensive at first, but I framed it as a team issue, not a personal one. We agreed on the deadline. The last two weeks ran smoothly. Lesson: address it early, address it privately, frame it as shared goals — not blame.'

■ ***Tip:*** *Use the 'shared goal' framing. Strong communication signal.*

Behavioral

Q8. Tell me about a time you took initiative.

A: Sample: 'At Delta, I noticed our team was manually checking PR statuses, JIRA tickets, and milestone progress in three separate dashboards. It took 15 minutes every morning just to know where we were. I built a status dashboard in HTML + JavaScript that pulled from all three sources and auto-refreshed every 5 minutes. I deployed it to a shared server. Within a week, three team members were using it. By the end of the rotation, my mentor asked me to document it for the next intern. Lesson: don't wait for someone to ask you to fix the small annoyances — solve them and share. The dashboard wasn't in my job description, but it 10x'd the team's morning routine.'

■ ***Tip:*** *From your resume: 'Built a JSON-driven status dashboard tracking ticket status, PR flow, milestones.' Hits multiple notes — useful for behavioral too.*

Behavioral

Q9. Why should we hire you over other new grads?

A: Specific differentiators. 'Three reasons. First, the breadth: CUDA systems work plus applied ML at Delta plus NLP projects — most new grads have one of these, I have three. Second, I've already shipped to production. My Delta automations have been running every day for over a year — that's past the prototype phase. Third, I have the right mindset for new grad: I've learned to ask for help fast, profile before optimizing, write tests before shipping. I'm a year ahead of someone fresh out of internships because the Delta co-op was an 18-month commitment.'

■ ***Tip:*** *Specific differentiators. NEVER 'I'm a hard worker' or 'I'm passionate.' That's everyone.*

Behavioral

Q10. Describe a time you used data to make a decision.

A: CUDA project profiling. Sample: 'During my CUDA optimization, I was choosing between three potential optimizations: (1) reduce kernel launches, (2) improve memory coalescing, (3) load balance between GPU and CPU for sparse levels. I could've guessed which mattered most. Instead, I profiled. NVIDIA Nsight showed memory was 65% of the bottleneck, kernel launches 15%, load imbalance 20%. So I tackled memory first — biggest lever. Got from 4x to 9x. THEN load balancing — to 12x. Kernel launches I never bothered with — too small to matter. Lesson: profile before you optimize. Intuition lies. Data wins.'

■ ***Tip:*** *Profile-before-optimize is a forever-useful story. Engineers love this.*

Behavioral

Q11. How do you stay current technically?

A: 'Layered. Daily: HackerNews and the orange site, scan for what's breaking. Weekly: I pick one paper or blog post to read deeply — recently been working through Andrej Karpathy's neural net YouTube series. Quarterly: a hands-on project with a new tool — last quarter I built a small RAG system with LangChain and a local Llama. Per class: I treat coursework projects as a chance to use something I haven't — that's how I picked CUDA for GPU programming class instead of OpenCL. Following: I follow specific people on Twitter — Jeff Dean, Andrej Karpathy, Yann LeCun, Sebastian Raschka. Reading those threads is a better signal than a textbook for what's actually shipping.'

■ ***Tip:*** *Specifics beat generics. 'Karpathy's YouTube' lands better than 'I read books.' Name names.*

Behavioral

Q12. Where do you see yourself in 5 years?

A: 'Honest: I want to be a strong technical IC. Not a manager. By year 5, I'd hope to be a Senior Software Engineer at this company, owning a meaningful piece of a product or platform end-to-end. The work I'm proudest of right now is the Delta automation that's still running every day. I want to keep building that — except at a scale where what I ship affects millions of users. I'm especially interested in the intersection of systems and AI — where my CUDA, ML, and platform engineering experience all come together. That's where I want to grow.'

■ ***Tip:*** *NCG-appropriate: ambitious but realistic. Don't say 'CEO' or 'principal in 3 years.' Don't say 'I don't know.'*

Behavioral

Q13. Why did you choose computer science?

A: Pick a real moment. Sample: 'Honestly, it started in 9th grade when my older brother showed me how to write a Python script to scrape his favorite anime site for new episode releases. It blew my mind that 10 lines of code could replace a daily manual task. From there, every project I've done — the recommendation engine, the search engine, the CUDA solver — has been a variation on that feeling: code as leverage. CS is the closest thing to magic that's real.'

■ ***Tip:*** *Personal story beats generic 'I love technology.' Use a specific moment.*

Behavioral

Q14. What kind of work environment helps you do your best work?

A: 'Three things. Concentration time: I do my deepest work in 3-4 hour uninterrupted blocks — usually morning. So I appreciate teams that respect focus time, not constant Slack pings. Code review culture: I learn most from senior engineers reviewing my PRs and pushing me. So I value teams with rigorous review, not rubber-stamp approval. Ambitious goals: I'm most motivated when the work matters. My Delta automation work is interesting because it has real users; if I were polishing internal tooling no one uses, I'd struggle. So I want work with real customer impact.'

■ ***Tip:*** *Three crisp things > five wishy-washy. Each tied to a real preference. Senior signal.*

Round 9: Hiring Manager / Final Round (45-60 min)

Last round before offer. Talks with the engineering manager you'd report to (or sometimes director). Tests culture fit, alignment with team goals, your questions about the team. Lower-pressure technically — more of a two-way conversation. They're deciding if they want you on their team specifically, AND you're deciding if you want them as your manager.

HM

Q1. Tell me about yourself, focusing on what you'd bring to my team.

A: Reframe your standard pitch toward THIS team. Sample for NVIDIA inference team: 'I'm Shivani — I just finished my BS in CS at UC Riverside and I'm in the MS program now. My background is systems plus applied AI. My CUDA work taught me how GPUs actually achieve their speedups — memory hierarchy, warp execution, occupancy. My Delta co-op taught me how to ship production data pipelines that real teams rely on. And my ML projects gave me the model and data fluency that the inference world needs. To your team specifically: I want to work on the substrate that makes large model inference possible. That's exactly the intersection of my three strengths.'

■ ***Tip:*** *Tailor the closing to THE TEAM. Hiring manager wants to feel you specifically want them.*

HM

Q2. What kind of work do you most want to do day-to-day?

A: 'Mix of three things. (1) Hands-on coding — I love writing code. Don't take this away from me. (2) Code review — I learn the most when senior engineers push back on my PRs. I want a team that does this rigorously. (3) Cross-functional collaboration — at Delta, I work with engineers, data scientists, and ops folks. That breadth keeps me curious. The balance I want: 70% coding, 20% design/review, 10% collaboration. The thing I want to avoid: drowning in meetings or being stuck on one tiny corner of the system.'

■ ***Tip:*** *Specific percentages signal you've thought about it. Most NCG just say 'I want to code.'*

HM

Q3. What are your career goals?

A: 'Short term — first 2 years here: I want to learn from senior engineers. Get rigorous in code review, write production-quality code, contribute to real product. By end of year 2, I want to own a meaningful component independently. Medium term — years 3-5: become a Senior Engineer who other people come to for help on systems-and-AI problems. Long term — beyond 5 years: I'm drawn to a Staff or Principal IC path. Not management. I want to be the technical lead, not the people manager. Whether that's 8 years from now or 12, I want to keep coding.'

■ ***Tip:*** *Honest about IC path. Don't pretend to want management if you don't. Hiring managers respect clarity.*

HM

Q4. How do you handle feedback?

A: 'Directly. I prefer harsh-but-fair over sugar-coated. At LifeStages, my mentor reviewed my Flask API code and said 'this is way too coupled — your handler is doing database, validation, and business logic. Split it.' Stung at first. But she was right. I refactored, and the next PR was clean. Lesson: defensive reactions waste energy. Hear the feedback, sit with it, ask follow-ups if needed, then change. The senior engineers who help me grow most are the ones who don't hold back. I'd ask the same of you.'

■ ***Tip:*** *Sets expectations that you want real feedback. That's the hiring manager's job and they'll appreciate hearing it.*

HM

Q5. What's something you've learned recently that you're excited about?

A: 'Two things actually. First, retrieval-augmented generation. I built a small RAG system with LangChain over the holidays — gave a local Llama model access to a custom doc set. The mental model click moment was

understanding that LLMs are knowledge engines but not knowledge bases — they need external grounding. That changes how I'd design any LLM-based product. Second, on the systems side, I've been reading Brendan Gregg's flame graphs work. Profiling production with flame graphs is a totally different skill from writing code; you can find bottlenecks in 30 seconds that would take hours of instrumentation. Both feel like force multipliers.'

■ ***Tip:*** *Specific. Recent. Two examples (one ML, one systems) shows breadth. Naming Brendan Gregg = systems senior signal.*

HM

Q6. What did you like most about your Delta co-op?

A: 'The work was actually used. The dashboards I built and the BoM parser I wrote are running every day, affecting how decisions get made. That's rare for an intern position. The thing I appreciated most: my mentor treated me like an engineer, not an intern. She gave me real problems, reviewed my code seriously, and expected me to ship. The hardest co-op moments were also the best — being told my PR wasn't ready and needed real fixes. That's how you grow.'

■ ***Tip:*** *Positive about prior employer. Show what you VALUED in a previous mentor — signals what you want here.*

HM

Q7. Have you ever felt unmotivated? How did you push through?

A: 'Yes, twice during the CUDA project. The first time was after my naive parallel version was slower than CPU — I'd spent two weeks and was going backwards. I almost gave up and rewrote it sequentially. What helped: I posted the issue in the GPU programming Slack channel at school. A grad student replied within 20 minutes — 'you're thread-divergent, look at warp occupancy.' Took me 2 hours to verify, fix it, and get a 3x speedup. Lesson: when you're stuck and demotivated, ASK FOR HELP. The penalty of asking is far smaller than the cost of grinding alone. I keep this in mind at work too — if I'm blocked for more than an hour, I ask someone.'

■ ***Tip:*** *The 'ask for help fast' lesson is GOLD for new grads. Hiring managers love hearing it.*

HM

Q8. What would your previous mentor say is your greatest strength?

A: 'My Delta mentor's feedback in my mid-rotation review was: 'Shivani writes more tests than her code requires.' She framed it as a strength — every function I write has unit tests. Some teams would call that overkill; she called it engineering maturity. Honestly, I think it's a combination of perfectionism and having shipped a buggy report once. After that, I never wanted to be the person who broke things in production. So I test everything. That's my biggest strength: I take quality seriously from day one.'

■ ***Tip:*** *Specific, sourced, with a story behind it. Better than 'I'm a hard worker.'*

HM

Q9. Why do you want to work specifically on my team?

A: Tailor heavily to the team — research the product before this round. Sample: 'Three reasons. First, your team works on [SPECIFIC PRODUCT FROM JD]. That product specifically interests me because [HONEST REASON — features, scale, technical challenges]. Second, when I researched your team, I noticed [PUBLIC INFO — recent papers, blog posts, talks]. The way you approach [TOPIC] aligns with how I think about software. Third, the new grad role here gives me access to [WHAT MAKES THIS TEAM SPECIAL — mentorship, scope, scale]. I want to learn from your senior engineers.'

■ ***Tip:*** *Research the team's PUBLIC OUTPUT before this interview — blog posts, talks, papers, GitHub. Drop references.*

HM

Q10. What's your salary expectation?

A: If this didn't come up with the recruiter (or hiring manager wants to confirm): 'Based on levels.fyi and Glassdoor data for new-grad SWE roles at [Company], I'm seeing total comp ranges from $X to $Y depending on level. I'd target the middle of that range. The specifics depend on the full package — base, equity, signing bonus, location. I'd defer to your recruiter for the offer details.'

■ ***Tip:*** *Hiring managers usually don't handle comp negotiation, but they might ask. Bounce back to recruiter.*

HM

Q11. Do you have any concerns about the role or team?

A: 'Two things I want clarity on. First, the new grad ramp-up: what does the first 6 months look like? Is there a buddy/mentor program, or is it sink-or-swim? Second, the tech stack: I see [TECH FROM JD] is primary — does the team also use [RELATED TECH]? I want to understand where I'd be growing technically.' Avoid manufactured concerns. If you have real ones, share them.

■ ***Tip:*** *Questions about ramp-up + tech stack are appropriate for HM round. Don't bring up things like comp, RTO, or PTO here — those are recruiter.*

HM

Q12. Do you have any questions for me?

A: Always have 5+ ready. HM-appropriate questions: (1) 'What does success look like for a new grad joining your team in the first 6 months?' (2) 'How is the team structured — how many engineers, what's the seniority distribution?' (3) 'What's the biggest technical challenge the team is tackling this year?' (4) 'How does the team approach code review and design docs?' (5) 'What's the typical project size for a new grad — single-engineer or larger team?' (6) 'How do you think about new grad growth — what resources or sponsorship does your team provide?' (7) 'What's something about your team that wouldn't be obvious from the job description?'

■ ***Tip:*** *NEVER 'no questions.' Single biggest red flag. Ask 3-5 minimum. The LAST question can be the best — leave them with a moment to share something they're proud of.*

Round 10: Comprehensive Python Round (45-60 min)

Pure Python language knowledge — separate from algorithm coding. Tests: data types, mutability, scoping, decorators, generators, context managers, OOP, the GIL, exception handling, common libraries, idioms, and gotchas. Asked at almost every company that uses Python. Many of these are 'rapid-fire' style — show fluency, not just correctness.

Python

Q1. What's the difference between a list, tuple, and set in Python?

A: Mutability, ordering, uniqueness.

```
# LIST: mutable, ordered, allows duplicates
# - Use for: ordered sequences you'll modify
# - O(1) append, O(n) insert at front
# - Memory: ~more than tuple due to over-allocation for growth
lst = [1, 2, 3, 2]
lst.append(4)    # ok
lst[0] = 99      # ok

# TUPLE: immutable, ordered, allows duplicates
# - Use for: fixed records (coordinates, DB rows, return multiple values)
# - Can be a dict key (hashable if all elements hashable)
# - Slightly faster than list, smaller memory
tup = (1, 2, 3, 2)
# tup[0] = 99    # ERROR — immutable

# SET: mutable, unordered, no duplicates
# - Use for: membership tests, deduplication, set operations
# - O(1) average for in, add, remove
# - Elements must be hashable (so no list-of-lists in a set)
s = {1, 2, 3}    # set literal
s.add(4)
s.add(1)         # no-op (already in)

# FROZENSET: immutable set (hashable, can be dict key)
fs = frozenset([1, 2, 3])

# Quick test of membership performance:
# x in list  -> O(n)
# x in set   -> O(1) average
# x in tuple -> O(n)
```

■ ***Tip:*** *When asked 'list vs tuple,' add 'tuple is hashable so can be a dict key.' Most candidates miss this.*

Python

Q2. What's the difference between `is` and `==`?

A: Identity vs equality. The classic gotcha.

```
# == checks VALUE equality (calls __eq__)
# is checks IDENTITY (same object in memory; same id())

a = [1, 2, 3]
b = [1, 2, 3]
c = a

print(a == b)   # True — same values
print(a is b)   # False — different objects
print(a is c)   # True — c references the same object as a

# COMMON GOTCHA: small integer caching
x = 256
y = 256
print(x is y)   # True — Python caches -5 to 256

x = 257
y = 257
print(x is y)   # MAY be False — outside cache range
```

```
# Don't rely on this. Use == for value comparison.

# Correct use of `is`:
# - Comparing to None: `if x is None:` (faster, idiomatic)
# - Comparing to singletons: True, False, NotImplemented, Ellipsis
# - Sentinel pattern:
SENTINEL = object()
def fn(arg=SENTINEL):
    if arg is SENTINEL:
        ...

# WRONG:
# if name == None:   <- works but slower & non-idiomatic
# Use:
# if name is None:
```

■ ***Tip:*** *ALWAYS use `is None`, never `== None`. PEP 8 standard. Interviewers notice.*

Python

Q3. What's mutable vs immutable? Why does it matter?

A: Whether the object's state can change after creation.

```
# IMMUTABLE: int, float, str, tuple, frozenset, bytes
# - Cannot be modified in place
# - Operations return NEW objects
# - Hashable -> can be dict keys, set members

# MUTABLE: list, dict, set, bytearray, custom classes (by default)
# - Modified in place
# - Operations may return None (e.g., list.sort())
# - Generally NOT hashable -> can't be dict keys

# CLASSIC BUG: mutable default argument
def append_item(item, lst=[]):   # DON'T DO THIS
    lst.append(item)
    return lst

print(append_item(1))   # [1]
print(append_item(2))   # [1, 2] — surprise! Shared list across calls!
print(append_item(3))   # [1, 2, 3]

# Why: default arg evaluated ONCE at function definition.
# All calls share that same list object.

# FIX:
def append_item(item, lst=None):
    if lst is None:
        lst = []
    lst.append(item)
    return lst

# Why immutability matters for hashability:
d = {}
# d[[1,2]] = "a"     # TypeError: unhashable type: 'list'
d[(1, 2)] = "a"      # OK — tuple is immutable, hashable

# Strings are immutable:
s = "hello"
s = s + " world"     # creates NEW string, doesn't modify
# Implications: don't build strings with += in a loop (O(n^2))
# Use '.join(list_of_strings) instead (O(n))
```

■ ***Tip:*** *Mutable default arg bug is asked at almost every Python interview. Always know the fix.*

Python

Q4. Explain Python's GIL. Why does it exist? What are the implications?

A: Global Interpreter Lock — only one thread executes Python bytecode at a time.

```
# WHAT: The GIL is a mutex in CPython that prevents multiple native
# threads from executing Python bytecode simultaneously.

# WHY:
```

```
# - CPython memory management (reference counting) isn't thread-safe
# - GIL is the simple solution: serialize bytecode execution
# - Removing it would require massive rework of the interpreter
#   (work is underway: PEP 703, optional no-GIL in 3.13+)

# IMPLICATIONS:
# - CPU-bound multi-threaded Python = NO speedup from multiple cores
# - I/O-bound multi-threaded Python = WORKS because GIL is released
#   during I/O (file, network, time.sleep)

# CONSEQUENCES — choose the right concurrency tool:

# 1. CPU-BOUND: use multiprocessing (separate processes, separate GILs)
from multiprocessing import Pool
with Pool(4) as p:
    results = p.map(cpu_heavy_function, large_list)

# 2. I/O-BOUND: use threading or asyncio
from concurrent.futures import ThreadPoolExecutor
with ThreadPoolExecutor(max_workers=10) as ex:
    futures = [ex.submit(fetch_url, url) for url in urls]

# 3. ASYNC: use asyncio (single thread, cooperative scheduling)
import asyncio
async def fetch_all(urls):
    tasks = [fetch_url_async(u) for u in urls]
    return await asyncio.gather(*tasks)

# 4. C EXTENSIONS: NumPy, Pandas, etc. release the GIL in their C code,
#    so you can get parallelism for heavy numerical work even from threads.

# QUICK MENTAL MODEL:
# - "I'm parsing big files / training a model" -> CPU-bound -> multiprocessing
# - "I'm calling 100 APIs" -> I/O-bound -> asyncio or threading
# - "I'm doing matrix math in NumPy" -> NumPy handles parallelism, just use it
```

■ ***Tip:*** *Senior signal: mention PEP 703 (no-GIL Python in 3.13+) — shows you follow language evolution.*

Python

Q5. What are *args and **kwargs?

A: Variable-length argument lists.

```
# *args: collects POSITIONAL args into a tuple
def my_func(*args):
    print(args)             # tuple of all positional args
    print(type(args))       # <class 'tuple'>

my_func(1, 2, 3)            # args = (1, 2, 3)

# **kwargs: collects KEYWORD args into a dict
def my_func(**kwargs):
    print(kwargs)           # dict of keyword args
    print(type(kwargs))     # <class 'dict'>

my_func(a=1, b=2)           # kwargs = {'a': 1, 'b': 2}

# Combined — common signature:
def my_func(required, default=None, *args, **kwargs):
    print(required, default, args, kwargs)

my_func("hi", "there", 1, 2, 3, x=10, y=20)
# required='hi', default='there', args=(1,2,3), kwargs={'x':10,'y':20}

# UNPACKING (the reverse):
def add(a, b, c):
    return a + b + c

values = [1, 2, 3]
print(add(*values))         # unpacks to add(1, 2, 3)

config = {'a': 1, 'b': 2, 'c': 3}
print(add(**config))        # unpacks to add(a=1, b=2, c=3)
# PRODUCTION PATTERN: wrapping/decorating
```

```
def log_calls(func):
    def wrapper(*args, **kwargs):
        print(f'Calling {func.__name__}({args}, {kwargs})')
        result = func(*args, **kwargs)
        print(f'  -> {result}')
        return result
    return wrapper

@log_calls
def add(a, b):
    return a + b

add(2, 3)
# Calling add((2, 3), {})
#   -> 5
```

■ ***Tip:*** **args and **kwargs are pure forwarding patterns. Use them when WRAPPING other functions; don't sprinkle them everywhere.*

Python

Q6. What's a decorator? Write one.

A: A function that takes a function, returns a modified function.

```
# Simple timer decorator
import time
from functools import wraps

def timer(func):
    @wraps(func)   # preserves func.__name__, __doc__, etc.
    def wrapper(*args, **kwargs):
        start = time.time()
        result = func(*args, **kwargs)
        elapsed = time.time() - start
        print(f'{func.__name__} took {elapsed:.3f}s')
        return result
    return wrapper

@timer
def slow_function():
    time.sleep(1)
    return "done"

slow_function()
# slow_function took 1.001s

# Equivalent syntax:
# slow_function = timer(slow_function)

# DECORATOR WITH ARGUMENTS:
def retry(max_attempts=3):
    def decorator(func):
        @wraps(func)
        def wrapper(*args, **kwargs):
            last_exc = None
            for attempt in range(max_attempts):
                try:
                    return func(*args, **kwargs)
                except Exception as e:
                    last_exc = e
            raise last_exc
        return wrapper
    return decorator

@retry(max_attempts=5)
def flaky_api_call():
    ...

# CLASS DECORATOR:
class CallCounter:
    def __init__(self, func):
        self.func = func
        self.count = 0
    def __call__(self, *args, **kwargs):
```

```
        self.count += 1
        print(f'Call #{self.count}')
        return self.func(*args, **kwargs)

@CallCounter
def greet(name):
    return f'Hello {name}'

greet('Alice')   # Call #1
greet('Bob')     # Call #2

# COMMON BUILT-IN DECORATORS:
# @property              — turn method into attribute access
# @classmethod           — method receives class, not instance
# @staticmethod          — method takes neither self nor cls
# @functools.lru_cache   — memoize function results
# @functools.wraps       — preserve metadata in wrapper
# @dataclass             — auto-generate __init__, __repr__, __eq__
```

■ ***Tip:*** *ALWAYS use @wraps in your decorators. Without it, the wrapped function loses its name and docstring.*

Python

Q7. What's a generator? Difference from a list?

A: Lazy iteration — produces values on demand.

```
# GENERATOR FUNCTION: uses yield instead of return
def count_up_to(n):
    for i in range(n):
        yield i

# Generator object created — code does NOT run yet
gen = count_up_to(5)
print(type(gen))         # <class 'generator'>

# Values produced one at a time as you iterate
for value in gen:
    print(value)         # 0 1 2 3 4

# Once exhausted, can't restart
list(gen)                # [] (empty)

# COMPARISON:
# List comprehension:
squares_list = [x**2 for x in range(1_000_000)]   # all in memory NOW
# Generator expression:
squares_gen = (x**2 for x in range(1_000_000))    # nothing computed yet

# Memory: list = 1M ints * ~28 bytes = ~28 MB
# Memory: generator = ~100 bytes (just state)

# When to use generators:
# 1. Large/infinite sequences (reading huge files)
def read_large_file(path):
    with open(path) as f:
        for line in f:        # file iteration IS a generator
            yield line.strip()

# 2. Pipelining transformations
def squared(nums):
    for n in nums:
        yield n * n

def evens(nums):
    for n in nums:
        if n % 2 == 0:
            yield n

# Lazy pipeline — no intermediate lists
nums = range(1_000_000)
result = sum(evens(squared(nums)))   # computes one at a time

# 3. Async-ish patterns: yield gives control back to caller
# GENERATOR INTERNALS:
```

```
# - Each yield pauses the function, returns a value
# - State (locals, line position) preserved in generator object
# - Next call resumes from where it paused
# - StopIteration raised when function returns or ends

# yield from — delegates to another generator
def chain(*iterables):
    for it in iterables:
        yield from it

list(chain([1, 2], [3, 4], [5]))    # [1, 2, 3, 4, 5]
```

■ ***Tip:*** *Generators are perfect for log processing, streaming ML data, large file parsing. Mention them when discussing memory efficiency.*

Python

Q8. What's a context manager? Why use one?

A: Object that manages setup/teardown around a `with` block.

```
# WHY: ensure cleanup even when exceptions happen.
# Classic example: file handles.

# WITHOUT context manager:
f = open('file.txt')
try:
    data = f.read()
finally:
    f.close()    # MUST close, or file descriptor leaks

# WITH context manager — equivalent, cleaner:
with open('file.txt') as f:
    data = f.read()
# f.close() called automatically, even on exception

# CREATING YOUR OWN: implement __enter__ and __exit__

class Timer:
    def __enter__(self):
        import time
        self.start = time.time()
        return self           # value bound by 'as'

    def __exit__(self, exc_type, exc_val, exc_tb):
        import time
        elapsed = time.time() - self.start
        print(f'Elapsed: {elapsed:.3f}s')
        return False          # False -> re-raise exception
                              # True  -> suppress exception

with Timer():
    expensive_operation()

# SHORTCUT: @contextmanager decorator
from contextlib import contextmanager

@contextmanager
def timer():
    import time
    start = time.time()
    try:
        yield                 # everything before = __enter__
                              # everything after = __exit__
    finally:
        elapsed = time.time() - start
        print(f'Elapsed: {elapsed:.3f}s')

with timer():
    expensive_operation()

# COMMON USE CASES:
# - File handling: with open(...)
# - Locks: with lock:
# - DB transactions: with conn:
```

```
# - Temp directories: with tempfile.TemporaryDirectory() as path:
# - Mocking in tests: with patch('module.func') as mock:
# - Suppressing exceptions: with contextlib.suppress(FileNotFoundError):

# MULTIPLE CONTEXTS in one with:
with open('a.txt') as fa, open('b.txt') as fb:
    ...

# NESTED is fine too:
with open('a.txt') as fa:
    with open('b.txt') as fb:
        ...
```

■ ***Tip:*** *Mention contextlib.suppress and @contextmanager — senior Python signals.*

Python

Q9. Explain Python's scoping rules — LEGB.

A: Local → Enclosing → Global → Built-in lookup order.

```
# LEGB:
# L - Local: inside the current function
# E - Enclosing: any outer function (nested functions)
# G - Global: module-level
# B - Built-in: Python's pre-defined names (print, len, etc.)

x = 'global x'

def outer():
    x = 'enclosing x'
    def inner():
        x = 'local x'
        print(x)        # 'local x' — found in L
    inner()
    print(x)            # 'enclosing x' — back to E

outer()
print(x)                # 'global x' — back to G

# COMMON GOTCHA: assigning makes a name LOCAL
x = 10
def my_func():
    x += 1              # UnboundLocalError!
    # Python sees the assignment, treats x as local,
    # but never initialized -> error.

# FIX 1: global keyword
def my_func():
    global x
    x += 1

# FIX 2: nonlocal keyword (for enclosing scope)
def outer():
    x = 10
    def inner():
        nonlocal x      # refer to outer's x, not global
        x += 1
    inner()
    print(x)            # 11

# CLOSURES: inner function captures enclosing variables
def make_counter():
    count = 0
    def increment():
        nonlocal count
        count += 1
        return count
    return increment

counter = make_counter()
print(counter())        # 1
print(counter())        # 2
print(counter())        # 3
# Each call to make_counter() makes a new closure with its own count.
```

```
# CLASSIC INTERVIEW GOTCHA — late binding:
funcs = [lambda: i for i in range(3)]
for f in funcs:
    print(f())          # 2 2 2  (NOT 0 1 2!)
# All lambdas capture the SAME i, which is 2 after the loop.

# FIX: default arg trick (evaluated at def time)
funcs = [lambda i=i: i for i in range(3)]
for f in funcs:
    print(f())          # 0 1 2  (correct)
```

■ ***Tip:*** *Late-binding gotcha is asked everywhere. Know the fix: default arg captures the value.*

Python

Q10. Difference between deep copy and shallow copy?

A: Reference vs recursive copy.

```
import copy

# SHALLOW COPY: new outer object, same inner references
original = [[1, 2], [3, 4]]
shallow = copy.copy(original)         # or list(original), or original[:]

shallow[0].append(99)                 # MUTATES the shared inner list
print(original)                       # [[1, 2, 99], [3, 4]] — affected!
print(shallow)                        # [[1, 2, 99], [3, 4]]

# DEEP COPY: new outer AND recursively new inner objects
original = [[1, 2], [3, 4]]
deep = copy.deepcopy(original)

deep[0].append(99)                    # ONLY mutates the deep copy's list
print(original)                       # [[1, 2], [3, 4]] — unchanged
print(deep)                           # [[1, 2, 99], [3, 4]]

# WAYS TO SHALLOW-COPY:
# - copy.copy(obj)
# - list(some_list)
# - some_list[:]
# - {**some_dict}
# - some_dict.copy()
# - set(some_set)

# WHEN TO USE EACH:
# Shallow: when you have immutable inner elements (strings, ints, tuples)
#   or when you WANT shared references
# Deep:    when inner objects are mutable AND you want full isolation

# PERFORMANCE: deep copy is slow for big structures.
# Often it's better to:
# - Use immutable data structures (tuples, frozensets, namedtuples)
# - Use dataclasses with frozen=True
# - Rebuild rather than copy (functional style)

# GOTCHA: assignment is NOT a copy
a = [1, 2, 3]
b = a               # b is the SAME object
b.append(4)
print(a)            # [1, 2, 3, 4] — mutated through b
```

■ ***Tip:*** *If asked: 'How do you copy a list?' — say `lst.copy()` or `lst[:]` (shallow). Most candidates say `=` which is wrong.*

Python

Q11. Explain Python's iterators and the iteration protocol.

A: Two methods: __iter__ and __next__.

```
# Iteration protocol:
# - iter(obj) calls obj.__iter__() -> returns an iterator
# - iterator must implement __next__() -> returns next value, raises StopIteration when done

# When you write `for x in collection:`, Python translates to:
```

```
it = iter(collection)
while True:
    try:
        x = next(it)
        # body
    except StopIteration:
        break

# CUSTOM ITERATOR CLASS:
class CountDown:
    def __init__(self, start):
        self.current = start

    def __iter__(self):
        return self            # we ARE our own iterator

    def __next__(self):
        if self.current <= 0:
            raise StopIteration
        self.current -= 1
        return self.current + 1

for n in CountDown(3):
    print(n)              # 3 2 1

# ITERABLE vs ITERATOR — distinction matters:
# - Iterable: has __iter__ (or __getitem__). Lists, tuples, strings, dicts.
# - Iterator: has __next__ AND __iter__ returning self.
# Lists are iterable but NOT iterators. iter(list) gives you the iterator.

# GENERATOR = simpler way to make an iterator
def count_down(start):
    while start > 0:
        yield start
        start -= 1

for n in count_down(3):
    print(n)              # 3 2 1

# ITERTOOLS — must-know library:
from itertools import chain, count, cycle, islice, takewhile

# chain: concatenate iterables
list(chain([1, 2], [3, 4]))             # [1, 2, 3, 4]

# count: infinite counter
list(islice(count(10), 5))              # [10, 11, 12, 13, 14]

# cycle: repeat forever
list(islice(cycle([1, 2, 3]), 7))       # [1, 2, 3, 1, 2, 3, 1]

# takewhile: stop when predicate fails
list(takewhile(lambda x: x < 5, [1, 4, 6, 4, 1]))   # [1, 4]

# zip on iterables of different lengths -> stops at shortest
list(zip([1, 2, 3], ['a', 'b']))        # [(1, 'a'), (2, 'b')]

# zip_longest -> pads
from itertools import zip_longest
list(zip_longest([1, 2, 3], ['a', 'b'], fillvalue=None))
# [(1, 'a'), (2, 'b'), (3, None)]
```

■ ***Tip:*** *itertools is the most underrated stdlib module. Knowing chain, islice, groupby, accumulate is senior signal.*

Python

Q12. What's the difference between __init__, __new__, and __call__?

A: Construction lifecycle and callability.

```
# __new__: CREATES the instance (returns it).
# Called BEFORE __init__. Receives the CLASS, returns the instance.
# Rarely overridden — main use is for immutable types or singletons.

# __init__: INITIALIZES the already-created instance.
```

```
# Receives the instance (self), returns None.
# What you usually override.

# __call__: makes the INSTANCE itself callable.
# Different lifecycle thing — invoked when you call obj()

class Person:
    def __new__(cls, *args, **kwargs):
        print(f'1. __new__ called on {cls}')
        instance = super().__new__(cls)
        return instance

    def __init__(self, name):
        print(f'2. __init__ called with name={name}')
        self.name = name

    def __call__(self, greeting):
        print(f'3. __call__: {greeting}, {self.name}')

p = Person("Alice")
# 1. __new__ called on <class '__main__.Person'>
# 2. __init__ called with name=Alice

p("Hello")
# 3. __call__: Hello, Alice

# SINGLETON via __new__:
class Singleton:
    _instance = None
    def __new__(cls):
        if cls._instance is None:
            cls._instance = super().__new__(cls)
        return cls._instance

a = Singleton()
b = Singleton()
print(a is b)          # True — same instance

# Callable instances are useful for:
# - Decorators (decorator-with-state via class)
# - Strategy pattern
# - Functions with state
# - Models (PyTorch's nn.Module makes instances callable):

import torch.nn as nn
class MyModel(nn.Module):
    def forward(self, x):
        return x * 2

model = MyModel()
output = model(input)   # __call__ invokes forward()
```

■ ***Tip:*** *Mention PyTorch's nn.Module — connects Python OOP to your ML world.*

Python

Q13. What's `self`? Why is it explicit in method signatures?

A: The instance reference, explicit by Python's design.

```
class Dog:
    def __init__(self, name):
        self.name = name        # `self` is the instance being created

    def bark(self):
        print(f'{self.name} says woof!')

d = Dog('Rex')
d.bark()                        # 'Rex says woof!'

# Behind the scenes, Python calls:
# Dog.bark(d)  ->  passes d as self

# WHY EXPLICIT?
# Python's philosophy: explicit is better than implicit.
```

```
# In Java/C++, `this` is implicit but causes confusion:
#   - Is this variable instance or local?
#   - Easy to forget `this.x = x` and accidentally shadow.
# Python: you always SEE which variables belong to the instance.

# self is just a naming convention — not a keyword.
# You CAN name it anything, but everyone uses 'self'.
class Bad:
    def method(banana):
        banana.x = 10           # legal but weird

# `cls` is the convention for class methods (receives the class):
class Counter:
    count = 0

    @classmethod
    def increment(cls):
        cls.count += 1

    @staticmethod
    def add(a, b):              # no self, no cls
        return a + b

# WHEN TO USE:
# Regular method (self):  needs instance state
# @classmethod (cls):     alternative constructors, factory methods
# @staticmethod:          utility function logically grouped with class

class Date:
    def __init__(self, year, month, day):
        self.year = year
        ...

    @classmethod
    def from_string(cls, date_str):
        """Alternative constructor."""
        y, m, d = date_str.split('-')
        return cls(int(y), int(m), int(d))

    @staticmethod
    def is_valid(year, month, day):
        return 1 <= month <= 12 and 1 <= day <= 31

d = Date.from_string('2025-12-31')
```

■ ***Tip:*** *Be ready: 'When use @classmethod vs @staticmethod?' classmethod = factory (uses cls). staticmethod = utility (uses neither).*

Python

Q14. Explain Python's exception hierarchy and how to handle exceptions properly.

A: Tree rooted at BaseException; catch narrow, not broad.

```
# HIERARCHY (simplified):
# BaseException
# +-- SystemExit                   (sys.exit() — usually don't catch)
# +-- KeyboardInterrupt            (Ctrl+C — usually don't catch)
# +-- Exception                    (almost everything inherits from here)
#     +-- ArithmeticError
#     |   +-- ZeroDivisionError
#     |   +-- OverflowError
#     +-- LookupError
#     |   +-- IndexError
#     |   +-- KeyError
#     +-- OSError                  (file/network errors)
#     |   +-- FileNotFoundError
#     |   +-- PermissionError
#     |   +-- ConnectionError
#     +-- TypeError
#     +-- ValueError
#     +-- RuntimeError
#     +-- StopIteration
#     ...
```

```
# CATCH SPECIFIC, NOT BROAD:
# Bad — catches EVERYTHING including KeyboardInterrupt:
try:
    risky()
except:               # never use bare except
    pass

# Bad — too broad:
try:
    risky()
except Exception:
    pass

# Good — catch what you actually expect:
try:
    risky()
except FileNotFoundError:
    handle_missing_file()
except PermissionError:
    handle_permission()

# RAISE properly:
def divide(a, b):
    if b == 0:
        raise ValueError(f'Cannot divide by zero: {a}/{b}')
    return a / b

# CUSTOM EXCEPTIONS:
class MyAppError(Exception):
    """Base class for all our exceptions."""
    pass

class ConfigError(MyAppError):
    pass

class ValidationError(MyAppError):
    def __init__(self, field, message):
        self.field = field
        self.message = message
        super().__init__(f'{field}: {message}')

# Catch the base class to handle all subclasses:
try:
    process()
except MyAppError as e:
    log.error(e)

# RE-RAISE preserving traceback:
try:
    process()
except Exception as e:
    log.exception('failed')
    raise                          # re-raises with original traceback

# CHAIN exceptions (3.x):
try:
    process()
except IOError as e:
    raise MyAppError('failed') from e   # original cause preserved

# `try/except/else/finally`:
try:
    f = open('x.txt')
except FileNotFoundError:
    print('not found')
else:
    # ONLY runs if no exception in try
    data = f.read()
finally:
    # ALWAYS runs (cleanup)
    if 'f' in locals() and not f.closed:
        f.close()
```

■ ***Tip:*** *Bare `except:` catches KeyboardInterrupt — won't let you Ctrl+C. Always specify exception types. Senior signal.*

Python

Q15. What are dataclasses? When use them?

A: Decorator that auto-generates __init__, __repr__, __eq__, etc.

```
from dataclasses import dataclass, field
from typing import List

# BEFORE dataclasses:
class Point:
    def __init__(self, x: float, y: float):
        self.x = x
        self.y = y

    def __repr__(self):
        return f'Point(x={self.x}, y={self.y})'

    def __eq__(self, other):
        if not isinstance(other, Point):
            return NotImplemented
        return self.x == other.x and self.y == other.y

# Tons of boilerplate.

# WITH @dataclass:
@dataclass
class Point:
    x: float
    y: float

# Automatic __init__, __repr__, __eq__:
p = Point(1.0, 2.0)
print(p)                # Point(x=1.0, y=2.0)
print(Point(1, 2) == Point(1, 2))   # True

# OPTIONS:
@dataclass(frozen=True)     # immutable — can hash, use as dict key
class FrozenPoint:
    x: float
    y: float

@dataclass(order=True)      # adds __lt__, __le__, etc. for sorting
class Score:
    points: int
    name: str

# DEFAULT VALUES:
@dataclass
class Config:
    host: str = 'localhost'
    port: int = 8080
    retries: int = 3

# MUTABLE DEFAULTS — use field(default_factory=...)
@dataclass
class TeamRoster:
    name: str
    members: List[str] = field(default_factory=list)
    # NOT members: List[str] = []  -- error: mutable default

# POST-INIT for derived attributes:
@dataclass
class Rectangle:
    width: float
    height: float
    area: float = field(init=False)

    def __post_init__(self):
        self.area = self.width * self.height

r = Rectangle(3, 4)
```

```
print(r.area)           # 12

# WHEN TO USE:
# - Data containers (records, configs, DTOs)
# - Simple value objects
# - Anywhere you'd write a bunch of __init__/__repr__/__eq__ boilerplate

# WHEN NOT:
# - Complex behavior — use a regular class
# - When inheritance gets tangled — Pydantic or attrs may be cleaner

# RELATED:
# - typing.NamedTuple: similar but immutable, inherits from tuple
# - Pydantic BaseModel: data validation + serialization (use in APIs)
```

■ ***Tip:*** *Mention Pydantic as the 'production' next step for data validation. FastAPI uses it everywhere.*

Python

Q16. What's the difference between `range`, `xrange`, and lists?

A: Python 2 vs 3 thing — but interviewers ask about iteration efficiency.

```
# Python 2:
# range(n) -> creates a list [0, 1, 2, ..., n-1]  (memory)
# xrange(n) -> creates a generator-like object (lazy)

# Python 3:
# range(n) -> creates a range OBJECT (lazy, like Python 2's xrange)
# xrange doesn't exist

# A range object is LAZY but not exactly a generator:
r = range(1_000_000)
print(type(r))          # <class 'range'>
print(r[500])           # 500 — supports indexing
print(len(r))           # 1_000_000 — supports len
print(500 in r)         # True — supports membership (O(1) for range!)

# But iteration is needed to actually produce values:
for i in r:             # values produced one at a time
    if i > 5:
        break

# Memory: range(1M) uses ~50 bytes. list(range(1M)) uses ~28 MB.

# range supports start, stop, step:
list(range(10))             # [0, 1, 2, ..., 9]
list(range(1, 10))          # [1, 2, ..., 9]
list(range(1, 10, 2))       # [1, 3, 5, 7, 9]
list(range(10, 0, -1))      # [10, 9, ..., 1]

# When you SEE list(range(...)), ask: why? Most iteration doesn't need a list.

# Common patterns:
# - Loop: `for i in range(n):`  (no list needed)
# - Reverse: `for i in reversed(range(n)):`  (no list needed)
# - Concrete list of indices: `list(range(n))`  (need list)

# COMPARISON OF "LAZY-ISH" THINGS:
# range:        random access, len, in (O(1) for arithmetic ranges)
# generator:    forward-only iteration
# iterator:     forward-only iteration
# list:         eager, all in memory
```

■ ***Tip:*** *range membership test is O(1) for arithmetic ranges — most candidates assume O(n). Senior signal.*

Python

Q17. Explain `__slots__`.

A: Optimization for memory: no per-instance __dict__.

```
# Without __slots__: every instance has __dict__ for attributes
class Point:
    def __init__(self, x, y):
        self.x = x
```

```
        self.y = y

p = Point(1, 2)
p.z = 3                # works — can add new attrs anytime
print(p.__dict__)      # {'x': 1, 'y': 2, 'z': 3}

# With __slots__: attributes are fixed, no __dict__
class PointSlim:
    __slots__ = ('x', 'y')

    def __init__(self, x, y):
        self.x = x
        self.y = y

p = PointSlim(1, 2)
# p.z = 3              # AttributeError — z not in __slots__
# print(p.__dict__)    # AttributeError — no __dict__

# MEMORY SAVINGS: 30-50% per instance, depending on attribute count.
# For 1 million instances, can save hundreds of MB.

# WHY USE:
# - Hot path with millions of small objects (game engines, parsers,
#   geometric computations, ML feature objects)
# - Want to PREVENT typo bugs (p.xx = 1 instead of p.x = 1)

# WHY NOT:
# - Lose flexibility (can't add arbitrary attrs at runtime)
# - Doesn't work cleanly with multiple inheritance
# - Dataclass: must declare with __slots__ in Python 3.10+
#   (older versions need workarounds)

@dataclass(slots=True)        # Python 3.10+
class Point:
    x: float
    y: float

# REAL-WORLD: SQLAlchemy ORM, attrs library, many ML libraries use slots.
```

■ ***Tip:*** *Mention 'I'd use __slots__ for hot paths handling millions of small objects' — connects to your CUDA/parallel computing background.*

Python

Q18. How does Python's garbage collection work?

A: Reference counting + cyclic garbage collector.

```
# TWO MECHANISMS:

# 1. REFERENCE COUNTING (primary)
# Every object has a ref count.
# Each new reference increments, each removed decrements.
# When count hits 0, object is freed immediately.

import sys
a = []                          # refcount = 1
b = a                           # refcount = 2
print(sys.getrefcount(a))       # 3 (+1 for getrefcount's own ref)

del b                           # refcount = 1
del a                           # refcount = 0 -> freed

# PROS: deterministic, immediate cleanup
# CONS: doesn't catch cycles

# CYCLE PROBLEM:
a = []
b = [a]
a.append(b)                     # a and b reference each other
# Even after del a; del b, refcounts don't hit 0 because of the cycle.

# 2. CYCLIC GARBAGE COLLECTOR (gc module)
# Periodically scans for cycles and reclaims them.
# Three generations (gen 0, 1, 2) — young objects scanned more often.
```

```
import gc
gc.collect()                    # force a collection
gc.get_count()                  # (gen0, gen1, gen2) counts
gc.disable()                    # disable cyclic GC (refcount still works)
gc.enable()

# WHY DISABLE GC?
# - In performance-critical code (HPC, real-time)
# - Then manually gc.collect() at safe points

# COMMON PITFALL: ref count keeps things alive
# When you think an object is freed, it may not be:
def process(big_data):
    result = compute(big_data)
    # big_data still referenced here, taking memory
    do_more()                   # if this is slow, big_data lingers
    return result

# Fix: explicitly delete to free early
def process(big_data):
    result = compute(big_data)
    del big_data                # release reference, refcount drops
    do_more()
    return result

# OR: use weakref for caches that shouldn't keep objects alive
import weakref
cache = weakref.WeakValueDictionary()
# Entries auto-removed when no other refs exist
```

■ ***Tip:*** *Mention weakref — used in caches, ORM relationships. Senior signal.*

Python

Q19. What's the difference between modules, packages, and namespaces?

A: Single file vs directory of files vs logical grouping.

```
# MODULE: a single .py file
# math.py is a module — you can import math

# PACKAGE: a directory containing modules + __init__.py
# Example:
# mypackage/
#     __init__.py        <- makes it a package
#     module1.py
#     module2.py
#     subpackage/
#         __init__.py
#         submodule.py

# Importing:
import mypackage
from mypackage import module1
from mypackage.subpackage import submodule
from mypackage.subpackage.submodule import some_func

# __init__.py:
# - Can be empty (just marks as package)
# - Can run init code
# - Can define what `from package import *` exports:
__all__ = ['module1', 'module2']

# NAMESPACE PACKAGES (Python 3.3+):
# Directories WITHOUT __init__.py can still be packages.
# Useful for splitting one logical package across multiple directories.

# IMPORT INTERNALS:
# 1. Check sys.modules cache (module already loaded?)
# 2. Find module using sys.path (a list of directories)
# 3. Compile to bytecode (.pyc in __pycache__/)
# 4. Execute top-level code
# 5. Cache result in sys.modules

import sys
```

```
print(sys.path)             # search paths
print(sys.modules.keys())   # loaded modules

# CIRCULAR IMPORTS — common bug:
# a.py: from b import thing
# b.py: from a import other
# At least one needs to be a lazy import (inside a function)
# Or restructure to avoid the cycle (common: a shared types module)

# ABSOLUTE vs RELATIVE imports:
from mypackage.module1 import thing     # absolute
from . import module1                   # relative (from same package)
from ..subpackage import submodule      # relative (from parent package)

# Best practice: absolute imports, except in obvious local cases.
```

■ ***Tip:*** *Mention circular imports — extremely common in real codebases. Fix: lazy imports OR restructure.*

Python

Q20. What is duck typing?

A: 'If it walks like a duck and quacks like a duck, it's a duck.'

```
# Duck typing: behavior matters, not type.
# Python doesn't check types; it tries operations and catches failures.

class Duck:
    def quack(self):
        print('Quack!')

class Person:
    def quack(self):
        print('I am quacking like a duck')

def make_it_quack(thing):
    thing.quack()    # works for ANYTHING with quack() method

make_it_quack(Duck())     # Quack!
make_it_quack(Person())   # I am quacking like a duck

# CONTRAST WITH JAVA: would require both to implement a Quackable interface.

# PYTHON'S WAY:
# - Try the operation, handle if it fails (EAFP — Easier to Ask Forgiveness)
def add(a, b):
    return a + b
# Works for: ints, floats, strings, lists, anything that defines __add__

# vs (LBYL — Look Before You Leap):
def add(a, b):
    if isinstance(a, (int, float)) and isinstance(b, (int, float)):
        return a + b
    # Limits what works.

# MODERN PYTHON: optional type hints + Protocols (PEP 544)
from typing import Protocol

class Quackable(Protocol):
    def quack(self) -> None: ...

def make_it_quack(thing: Quackable) -> None:
    thing.quack()

# Structural typing — anything with quack() method satisfies Quackable.
# mypy / pyright check this at type-check time without inheritance.

# TYPE HINTS DON'T ENFORCE AT RUNTIME:
def add(a: int, b: int) -> int:
    return a + b

add('hello', 'world')   # Works! Returns 'helloworld'. mypy would catch.

# Use typing for documentation + static analysis. NOT runtime safety.
# Use pydantic / dataclass with runtime validation for that.
```

■ ***Tip:*** *Mention Protocols (PEP 544) — modern Python structural typing. Senior signal.*

Python

Q21. Explain async/await.

A: Cooperative concurrency — single thread, multiple tasks.

```
# ASYNCIO: concurrency without threads.
# - One event loop
# - Tasks await on I/O (or other awaitables)
# - Loop schedules other tasks while one is waiting

import asyncio
import aiohttp

# An async function (coroutine):
async def fetch_url(url):
    async with aiohttp.ClientSession() as session:
        async with session.get(url) as response:
            return await response.text()
# Calling it doesn't run — it creates a coroutine object.

# To run, need an event loop:
async def main():
    # Run sequentially (slow):
    a = await fetch_url('https://example.com/a')
    b = await fetch_url('https://example.com/b')

    # Run concurrently (fast — both fetched at same time):
    a, b = await asyncio.gather(
        fetch_url('https://example.com/a'),
        fetch_url('https://example.com/b'),
    )

asyncio.run(main())

# When does async help?
# - I/O-bound (network, file, DB) — YES, big speedup
# - CPU-bound — NO, GIL still blocks
# - Mixed — use loop.run_in_executor for CPU parts

# COMMON GOTCHAS:

# 1. Forgetting await
async def bad():
    fetch_url(...)        # creates coroutine, doesn't run it!
                          # warning: 'coroutine was never awaited'

# Fix:
async def good():
    await fetch_url(...)

# 2. Mixing sync and async
async def mixed():
    response = requests.get(url)   # BLOCKS the event loop!
# Use the async version (aiohttp, httpx) instead.

# 3. CPU work in async
async def slow():
    await asyncio.sleep(0)
    big_computation()     # blocks for seconds — bad

# Fix: offload to thread pool
async def fixed():
    loop = asyncio.get_event_loop()
    await loop.run_in_executor(None, big_computation)

# MODERN ALTERNATIVES:
# - trio: structured concurrency, cleaner than asyncio
# - anyio: works with both asyncio and trio
# - For new projects, asyncio is still the standard.
```

■ ***Tip:*** *If interviewer asks 'asyncio vs threading vs multiprocessing': asyncio for I/O-bound with many concurrent ops; threading for I/O-bound few ops or blocking libs; multiprocessing for CPU-bound.*

Python

Q22. What are Python's magic / dunder methods?

A: Double-underscore methods that hook into language operators.

```
# Magic methods make YOUR classes work with Python syntax.

class Vector:
    def __init__(self, x, y):
        self.x, self.y = x, y

    # __repr__ — unambiguous string for debugging
    def __repr__(self):
        return f'Vector({self.x}, {self.y})'

    # __str__ — user-friendly string
    def __str__(self):
        return f'({self.x}, {self.y})'

    # __eq__ — supports ==
    def __eq__(self, other):
        if not isinstance(other, Vector):
            return NotImplemented
        return self.x == other.x and self.y == other.y

    # __hash__ — required if __eq__ is defined and you want hashable
    def __hash__(self):
        return hash((self.x, self.y))

    # __add__ — supports +
    def __add__(self, other):
        return Vector(self.x + other.x, self.y + other.y)

    # __mul__ — supports *
    def __mul__(self, scalar):
        return Vector(self.x * scalar, self.y * scalar)

    # __len__ — supports len()
    def __len__(self):
        return 2

    # __getitem__ — supports indexing
    def __getitem__(self, idx):
        if idx == 0: return self.x
        if idx == 1: return self.y
        raise IndexError

    # __iter__ — supports for loops
    def __iter__(self):
        yield self.x
        yield self.y

    # __bool__ — supports truthiness
    def __bool__(self):
        return self.x != 0 or self.y != 0

    # __lt__, __le__, __gt__, __ge__ — comparison
    def __lt__(self, other):
        return self.magnitude() < other.magnitude()

    def magnitude(self):
        return (self.x**2 + self.y**2) ** 0.5

# Now Vector acts like a built-in:
v1 = Vector(1, 2)
v2 = Vector(3, 4)
print(v1 + v2)             # Vector(4, 6)
print(v1 * 3)              # Vector(3, 6)
print(v1 == Vector(1, 2))  # True
print(len(v1))             # 2
print(v1[0])               # 1
for c in v1: print(c)      # 1, 2
print(bool(v1))            # True
print(v1 < v2)             # True
```

```
# COMMON DUNDERS:
# __init__, __new__, __del__       construction/destruction
# __repr__, __str__                string representation
# __eq__, __hash__, __lt__, ...    comparison
# __add__, __sub__, __mul__, ...   arithmetic
# __getitem__, __setitem__         indexing
# __iter__, __next__               iteration
# __enter__, __exit__              context manager
# __call__                         callable
# __getattr__, __setattr__         attribute access
```

■ ***Tip:*** *If you override __eq__, you MUST decide on __hash__. Either define it or set __hash__ = None to make unhashable.*

Interview Day Workflows

Muscle-memory playbooks for typical interview situations. Memorize the structure; the specifics flex.

How to approach ANY coding problem

Universal 8-step flow

```
1. REPEAT the question back in your own words
    -> Confirms understanding AND buys thinking time
2. ASK clarifying questions:
    -> Input format? Sorted? Empty allowed? Negatives? Duplicates?
    -> Constraint sizes? (small n=5 -> brute force OK)
    -> Output format? Multiple valid answers?
3. WORK THROUGH an example by hand
    -> Pick small input, show how you'd solve manually
    -> Often reveals the algorithm
4. STATE your approach BEFORE coding
    -> 'I'm going to use a hash map to ...'
    -> Get interviewer agreement first
5. CODE clearly
    -> Type hints, descriptive names, no premature optimization
    -> Talk while you code
6. TEST on the example
    -> Walk through your code line by line
7. EDGE CASES: empty, single element, all duplicates, max size
8. COMPLEXITY: state time + space at the end
```

Stuck on a coding problem? UNSTICK yourself.

When the path forward is unclear

```
1. SLOW DOWN, don't panic
2. Restate the problem out loud
3. Try a smaller / simpler example
4. List similar problems you've seen — what pattern fits?
    -> Two pointers? Sliding window? Hash map? DP? BFS/DFS?
5. Brute force first — get ANY solution working
    -> O(n^2) > nothing
6. ASK for a hint: 'I'm thinking about X, am I on the right track?'
    -> Showing you can ask for help = better signal than silence
7. If still stuck, talk through WHAT you've tried
    -> Sometimes the interviewer redirects you
```

Behavioral question -> STAR

Tell me about a time...

```
S - Situation: 1 sentence context (what, where, when)
T - Task: what was YOUR responsibility?
A - Action: what did YOU do? (most time here)
    -> Use 'I', not 'we' (interviewer wants YOUR contribution)
R - Result: measurable outcome + what you learned

Have 5-7 stories ready, mapped to common categories:
    1. Challenge overcome    (CUDA optimization)
    2. Conflict resolved     (Delta XML parser disagreement)
    3. Team collaboration    (LifeStages Align app)
    4. Failure + lesson      (early Delta report bug)
    5. Initiative shown      (Delta status dashboard)
    6. Quick learning        (LangChain ramp-up)
    7. Feedback handled      (Flask refactor at LifeStages)
```

Most behavioral questions map to one of these.

System design (NCG style)

ARCH framework

```
A - Actors: who uses this? (users, admins, services)
R - Requirements:
    Functional: what must it DO?
    Non-functional: scale, latency, availability?
C - Components: draw boxes — be CONCRETE
    [Client] -> [LB] -> [API] -> [Cache] -> [DB]
    Don't be vague — name specific tech (Redis, Postgres, Kafka)
H - High availability:
    What fails? How recover?
    Where bottlenecks?
    Multi-region?
R - Risks / tradeoffs:
    Consistency vs availability
    Cost vs performance
    Complexity vs simplicity

FOR NCG: don't over-engineer. Better to have a clean simple
design that handles 10K users than a confused one for 1B.
```

AI/ML question approach

Walk through ML thinking

```
1. CLARIFY the problem type: classification / regression / clustering?
2. DATA: what's available? labeled? imbalanced? volume?
3. FEATURES: what would you use? how engineered?
4. MODEL choice: simple first (logistic regression, decision tree)
    -> Then complex (XGBoost, neural net) only if needed
5. EVALUATION:
    Classification: precision, recall, F1, ROC-AUC
    Regression: MSE, MAE, R^2
    Ranking: NDCG, MRR
6. TRAINING:
    Train/val/test split (or k-fold)
    Avoid leakage (no future info in features)
    Regularize to prevent overfitting
7. PRODUCTION:
    Latency? Batch vs real-time?
    Monitoring (drift, performance over time)?
    A/B test new models before full rollout?
```

Resume project deep-dive

When asked 'walk me through X'

```
Use the 6-part frame:
1. WHAT was the problem? (1 sentence)
2. WHY did it matter? (impact context)
3. APPROACH chosen + alternatives considered
4. KEY technical decisions (and trade-offs)
5. RESULT — measurable outcomes
6. WHAT you learned + what you'd do differently

Be ready to go DEEP on any project on resume.
If you wrote 'CUDA 12x speedup,' know:
    - WHY 12x not 100x? (memory bandwidth limit, kernel launch overhead)
    - What was the baseline? (sequential C++ BFS)
    - Profiler used? (NVIDIA Nsight)
    - What was the longest-pole bottleneck? (uncoalesced memory)
    - How would you scale beyond 1 GPU? (multi-GPU, NCCL)
```

Salary negotiation (when offer arrives)

Don't accept on the call

```
1. THANK them for the offer; ask for it in writing
2. Get DETAILS: base, RSU value, RSU vesting, signing, location
3. Ask for TIME: 'I'd like a few days to review with my family'
    -> Standard 5-7 days is reasonable
4. RESEARCH: levels.fyi, Glassdoor, Blind for this company / level
5. If you have competing offers, MENTION them (don't bluff)
6. COUNTER with specific number + reason:
    'Based on the role and market data, I'm seeing X for similar
    profiles. Can you match or come closer to that?'
7. NEGOTIABLE levers:
    - Base salary (usually 5-15% room)
    - Signing bonus (often easier to give than base)
    - RSU refresh schedule
    - Start date
    - Title / leveling
8. Once final, ACCEPT clearly and in writing

RULE: even if first offer feels great, ALWAYS counter at least once.
Top companies expect it. They've left margin in the offer.
```

What to do AFTER each interview

Post-interview routine

```
1. WRITE DOWN every question you got asked
    -> Build your personal interview question bank
    -> Helps prep for next round/company
2. NOTE what went well and what didn't
    -> Honest self-assessment
3. SEND a thank-you email within 24 hours
    -> Reference something specific from the conversation
    -> 'Thanks for the discussion on the inference platform —
        I went home and read up on Triton, which was a great pointer.'
4. DON'T ruminate. Whatever happens, happens.
    -> Start prep for next company
5. IF rejected: politely ask for feedback
    -> Half of recruiters will give it. That's gold for next time.
6. KEEP a tracker:
    Company | Date | Round | Outcome | Notes
    Lets you see patterns across applications.
```

Comprehensive Cheatsheet

Final morning review. Read THIS, don't try to learn new things the day-of. Frameworks, commands, mental models, company tips.

Answer Frameworks (use verbatim)

STAR (behavioral)	Situation → Task → Action → Result
ARCH (system design)	Actors → Requirements → Components → High availability → Risks
Coding problem (8 steps)	Repeat → Clarify → Example → State approach → Code → Test → Edge cases → Complexity
Project deep-dive (6 parts)	What → Why → Approach → Decisions → Result → Learned
Tool comparison	What it does → strengths → weaknesses → when I'd use it
Asked your weakness	Real weakness + how you actively manage it
Why this company	Specific product/team detail + how it ties to your background

LeetCode patterns (recognize in 30 seconds)

sorted array, find pair / palindrome	Two Pointers
contiguous subarray/substring property	Sliding Window (fixed or variable)
top k / kth largest / k closest	Heap (size k)
shortest path / level-by-level	BFS
all paths / count components / cycle	DFS or Union-Find
dependencies / build order	Topological Sort
all subsets / permutations / N-queens	Backtracking
min / max / count of ways	Dynamic Programming
matrix sorted rows/cols	Binary Search or staircase
prefix / autocomplete / word lookup	Trie
next greater / smaller element	Monotonic Stack
intervals overlap / merge	Sort + sweep
LRU / LFU cache	Doubly-linked list + hash map

Time complexity quick reference

Array index access	O(1)
Array append (amortized)	O(1)
Array insert at front	O(n)
Dict get/put (avg)	O(1)
Set add/in (avg)	O(1)
Heap push/pop	O(log n)
Sort	O(n log n) — Python uses Timsort
BFS / DFS	O(V + E)
Binary search	O(log n)
Dijkstra (binary heap)	O((V + E) log V)

Tree traversal	O(n)
Quick sort (avg / worst)	O(n log n) / O(n²)

Input size → acceptable complexity

n ≤ 10	O(n!) — permutations OK
n ≤ 20	O(2^n) — subsets OK
n ≤ 500	O(n^3)
n ≤ 5,000	O(n^2)
n ≤ 10^6	O(n log n) or O(n)
n ≤ 10^8+	O(n) or O(log n) only

Python essentials (memorize)

Counter / freq	from collections import Counter; Counter(it).most_common(N)
Default dict	from collections import defaultdict; defaultdict(list)
Stack	list — .append() and .pop()
Queue (both ends)	from collections import deque — .append() / .popleft()
Min-heap	import heapq — heappush(h, x), heappop(h)
Max-heap trick	heapq with negated values
Top K	heapq.nlargest(k, it) / nsmallest(k, it)
Binary search	import bisect — bisect_left / insort
Memoize	from functools import lru_cache — @lru_cache
Combinations / perms	from itertools import combinations, permutations, product
Running sum	from itertools import accumulate
Type hints	List, Dict, Tuple, Optional from typing
Walrus	if (m := match(p, s)) is not None: use(m)
Async run	asyncio.run(main()) — only at top level
Async parallel	await asyncio.gather(*tasks)

ML / AI essentials

Supervised vs unsupervised	labeled vs unlabeled data
Bias-variance tradeoff	underfit vs overfit
Train / val / test split	Typically 70/15/15 or 80/10/10
K-fold cross-validation	Use when data is small (k=5 or 10)
Classification metrics	precision, recall, F1, ROC-AUC, PR-AUC
Regression metrics	MSE, MAE, R^2, RMSE
Ranking metrics	NDCG, MRR, MAP
Regularization	L1 (sparse), L2 (small weights), dropout
Embedding	dense low-dim representation of high-dim input
Backprop =	chain rule applied layer by layer
Common optimizer	Adam (default), SGD + momentum
Cold start (recommenders)	new user → ask preferences; new item → use content features

Transfer learning	use pre-trained model, fine-tune for your task
LLM tokenization	BPE or SentencePiece sub-word units
Attention (informally)	each token can 'look at' all others, weighted

Company-specific signals to drop naturally

NVIDIA	CUDA, GPU memory hierarchy, warp execution, Tensor Cores, Triton inference
AMD	ROCm, MI300, HIP (CUDA equivalent), competitive vs NVIDIA
Google	Scale (Spanner, BigTable), Borg/K8s heritage, Gemini, MapReduce origins
Meta	PyTorch (Meta's), Llama, AI infra at scale, recommender systems
Microsoft	Azure, GitHub Copilot, OpenAI partnership, Windows + Linux dev
Apple	Privacy by design, on-device AI (Apple Intelligence), Metal, Swift
Netflix	Microservices, chaos engineering, Cassandra at scale, recommendation ML
Palo Alto Networks	Cortex XDR, threat intelligence, ML for security, zero trust
AppZen	AP automation, NLP for receipts/invoices, ML model lifecycle
Intuit	TurboTax ML, QuickBooks, GenAI for SMB workflows, financial AI
CrowdStrike	Falcon platform, EDR, threat hunting, behavioral ML, cloud-native
ServiceNow	Now Platform, Now Assist (GenAI), workflow automation, ITSM

Your project elevator pitches

CUDA BFS Maze Solver	Parallel BFS on 512x512 mazes, 12x speedup. Optimized: thread block sizing (256 best), memory coalescing (flat row-major), GPU+CPU hybrid for sparse levels. Profiled with Nsight.
Anime Recommender	Spark MLlib ALS + KMeans on 17K titles. Matrix factorization for user/item embeddings, KMeans for user clustering. Flask + React frontend. SQLite backend.
Reddit Search Engine	PyLucene + BM25 ranking + NLP preprocessing. Flask frontend. Compared retrieval algorithms. Production version would use Elasticsearch.
Delta LEAP BoM Parser	Parsed complex aircraft XML with BeautifulSoup, scikit-learn for categorization. Automated hierarchy generation across thousands of serialized parts. In daily production use.
LifeStages Align NLP	Flutter + Flask app. TextBlob for emotional tone categorization in journal entries. Senior version would fine-tune BERT on labeled mental wellness text.
KNN Feature Selection	Forward + backward feature selection algorithms. Validation-driven. Visualized feature importance + accuracy over selection iterations.
8-Puzzle Solver	Minimax + Alpha-Beta pruning. Heuristic evaluation. Compared search strategies on solve time + node count.

Interview day discipline

Repeat the question	Buys thinking time, confirms understanding
Think out loud	Reasoning is graded, not just final answer
Ask clarifying questions	Especially for system design — never assume requirements
When stuck	State what you DO know; partial credit > silence
Coding edge cases	Empty? Single element? Negative? Duplicates? Overflow?
Cite real stories	Your CUDA, Delta, anime recommender, search engine — use them
Always ask 3-5 questions	Never 'no I'm good' — biggest red flag
End strong	1 line about why you're excited specifically about this team

After interview	24hr thank-you note referencing something specific from chat
Sleep 8+ hours night before	Caffeine doesn't replace this
Don't ruminate after	Whatever happens, happens. Start prep for next round/company.

Resources to use BEFORE the interview

levels.fyi	Salary benchmarks by company/level/location
Glassdoor	Interview experience reports by company
Blind	Anonymous discussion of comp + interview process
LeetCode	Coding practice — focus on Top 150 + Patterns
NeetCode.io	Free curated LC practice by pattern
Pramp / Interviewing.io	Free mock interviews
System Design Primer (GitHub)	Free system design study guide
Cracking the Coding Interview	The classic — still relevant
Designing Data-Intensive Applications	For system design depth
Company engineering blogs	Read the team's published work before the interview

The single rule: every answer maps to a REAL project on your resume. CUDA 12x, Delta LEAP parser, ALS+KMeans recommender, PyLucene Reddit search, TextBlob Align — these are your gold. Use them. Good luck.